"You declare the desired state; Kubernetes makes it happen."

MODULE 42

Kubernetes (k3s) Deployment

Deploy, manage, and scale containerized applications with k3s -- and compare architectures with OpenShift.


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT


TEKsystems
Global Services
WORKFORCE
DEVELOPMENT

MODULE 42 OVERVIEW

Deploy, manage, and scale containerized applications with k3s -- and compare with OpenShift's enterprise Kubernetes platform.

This module moves the CRM platform from a single Docker container (Lab 41) to a real Kubernetes cluster -- Deployments, Services, ConfigMaps, Secrets, probes, rollouts, and rollbacks -- using k3s as the cohort's lightweight cluster runtime.

DURATION & LAB

1.5 Hours Theory

Kubernetes architecture, workloads, networking, config, rollouts, scaling, and k3s vs OpenShift.

2.5 Hours Lab

lab42-crm: Deployment, Service, ConfigMap/Secret, probes, Ingress, and a rollout + rollback drill.

Scenario

The crm-api container from Lab 41, deployed declaratively to the shared training k3s cluster.

MODULE ARC

Understand Kubernetes

Architecture: control plane, worker nodes, Pods, ReplicaSets, Deployments, Services.

Operate Safely

ConfigMaps/Secrets, rolling updates, rollbacks, scaling, and probes that keep traffic safe.

Compare & Build

k3s vs OpenShift, then deploy, expose, and rehearse rollback for the real lab42-crm lab.

KEY TAKEAWAY Total time: 4 hours (1.5 hours theory + 2.5 hours hands-on lab). By the end, you will deploy, expose, scale, and safely roll back the CRM API on a real Kubernetes (k3s) cluster.

WHY CONTAINER ORCHESTRATION MATTERS

Containers package applications and their dependencies, but running containers at scale introduces complexity -- orchestration brings order, automation, and resilience.

Challenges Without Orchestration	With Container Orchestration
Manual deployment -- time-consuming and error-prone across multiple servers.	Automated deployment -- consistent, repeatable rollouts with less manual effort.
Scaling complexity -- difficult to scale up or down based on demand.	Elastic scalability -- automatically scale applications based on real-time demand.
Poor resource utilization -- servers underutilized or overloaded.	Optimal resource utilization -- the right workload on the right resources.
Lack of resilience -- application failures can cause downtime.	High availability & self-healing -- automatically detects failures and recovers.
Operational overhead -- infrastructure, updates, networking, and security are complex.	Reduced operational overhead -- simplify infrastructure management.

BUSINESS IMPACT

Faster Time to Market

Deploy and update applications quickly and safely.

Lower Costs

Improve efficiency and reduce infrastructure waste.

Improved Reliability

Deliver consistent performance with higher uptime.

Better Agility

Adapt to changing business needs with ease.

Focus on Innovation

Spend less time on ops, more time on what matters.

KEY TAKEAWAY Orchestration doesn't just run containers -- it automates deployment, scaling, and recovery so teams ship faster, cut costs, and keep applications reliable.

Why Container Orchestration Matters

Challenges Without Orchestration

Manual Management	Deploying, scaling, and managing containers manually is time-consuming and error-prone.
Poor Resource Utilization	Resources are underutilized or over-provisioned, increasing costs.
Limited Scalability & Availability	Hard to scale applications and ensure high availability and resilience.
Complex Updates & Rollbacks	Updating and rolling back applications is risky and complicated.
Operational Overhead	High operational effort to maintain infrastructure and application lifecycle.

Container Orchestration

With Container Orchestration

Automated Management	Automates deployment, scaling, healing, and lifecycle management.
Optimal Resource Utilization	Efficient scheduling and resource allocation reduce waste and cost.
High Scalability & Availability	Auto-scaling, self-healing, and load balancing ensure always-on applications.
Seamless Updates & Rollbacks	Rolling updates and easy rollbacks minimize downtime and risk.
Reduced Operational Overhead	Simplifies operations and improves developer productivity and focus.

Key Benefits

Performance
Maintain desired state for optimal performance at all times.

Cost Efficiency
Better resource utilization leads to lower infrastructure and operational costs.

Reliability
Self-healing and redundancy ensure application reliability and uptime.

Scalability
Scale applications up or down automatically based on demand.

Faster Delivery
Automated deployments and rollbacks enable faster releases.

Developer Productivity
Developers focus on code, not infrastructure complexity.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to confidently deploy and operate containerized applications using k3s and understand how OpenShift extends Kubernetes for enterprise use.

1. **Kubernetes Fundamentals —** explain what Kubernetes is and describe its architecture and core components.
2. **Kubernetes Resources —** create and manage Pods, ReplicaSets, Deployments, Services, ConfigMaps, and Secrets.
3. **Deploy Applications —** deploy containerized applications using Deployments and understand the deployment workflow.
4. **Updates & Rollbacks —** perform rolling updates, rollbacks, and understand strategies for zero-downtime deployments.
5. **Scale Applications —** scale applications manually and understand the fundamentals of auto scaling.
6. **Understand k3s —** explain what k3s is, its features, and why it is ideal for lightweight environments.
7. **Compare k3s and OpenShift —** compare architecture, features, and use cases.
8. **Explore OpenShift Components —** understand Projects, Routes, Image Streams, and Build Configurations.
9. **Enterprise Operations —** implement high availability concepts, self-healing applications, and auto scaling fundamentals.
10. **Apply in a Lab —** deploy a real-world application on k3s and explore OpenShift features through hands-on labs.

KEY TAKEAWAY Gain practical skills to deploy, manage, and scale containerized applications using k3s and compare with OpenShift.

WHAT IS KUBERNETES?

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

IN SIMPLE TERMS

- Kubernetes takes care of the heavy lifting of running containers in production.
- So you can focus on building applications, not managing infrastructure.

KEY CHARACTERISTICS

Automates Deployment — deploys and updates applications consistently.

Manages Resources — optimizes compute, storage, and network use.

Scales Automatically — scales applications up or down based on demand.

Ensures High Availability — keeps applications running with minimal downtime.

Self-Heals — automatically detects and recovers from failures.

KEY CAPABILITIES

Container Orchestration — runs containers across a cluster of machines.

Declarative Configuration — define the desired state; Kubernetes ensures it.

Portability — run workloads anywhere -- on-prem, cloud, or hybrid.

Extensible Ecosystem — a large ecosystem of tools and integrations.

Observability — built-in support for logging, monitoring, health checks.

HOW IT WORKS

1. **You Define**
   Describe your app in YAML.
2. **Kubernetes Schedules**
   Places Pods on the right nodes.
3. **Runs & Manages**
   Runs containers, monitors health.
4. **Adapts Automatically**
   Scales, updates, recovers.
5. **Stays Available**
   High availability, no manual work.

KEY TAKEAWAY Kubernetes automates deployment, scaling, and recovery for containerized applications -- you declare the desired state, and Kubernetes makes it happen.

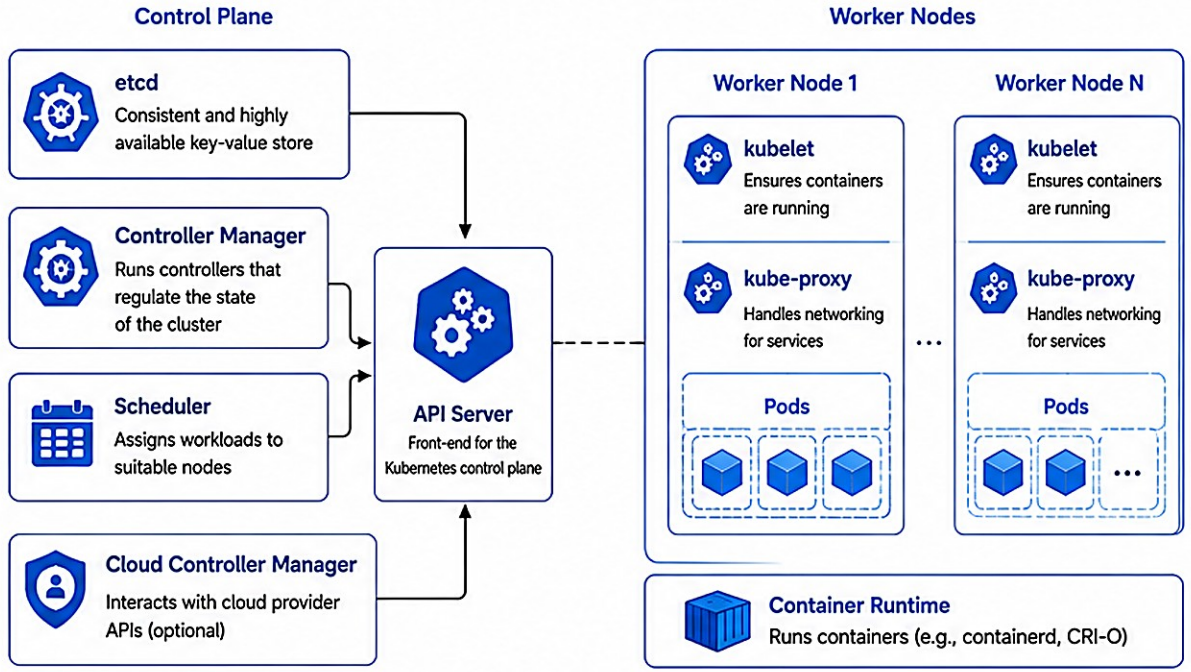
What is Kubernetes?

Definition

Kubernetes (K8s) is an open-source container orchestration platform that automates the deployment, scaling, and management of containerized applications.

kubernetes

Kubernetes Architecture



Key Features

- Automated Deployment and Rollouts
- Scalability and Elasticity
- Self-healing (Health Checks & Auto Recovery)
- Load Balancing and Service Discovery
- Storage Orchestration
- Secret and Configuration Management

Benefits

Speed
Faster deployments and continuous delivery

Efficiency
Optimal resource utilization and reduced costs

Reliability
High availability and automatic recovery

Portability
Run anywhere: on-prem, cloud, or hybrid

Extensibility
Rich ecosystem with plugins and integrations

Declarative
Define desired state and let Kubernetes manage it

KUBERNETES ARCHITECTURE OVERVIEW

Kubernetes follows a master-worker model to run, manage, and scale containerized applications.

THE MASTER-WORKER MODEL

	Control Plane (Master)	Worker Nodes
Role	Makes global decisions about the cluster (scheduling, detection, response).	Run and host your containerized applications.
Key parts	API Server, etcd, Scheduler, Controller Manager.	Kubelet, kube-proxy, Pods (containers).
Talks to	Users/Clients (kubectl, CLI, UI) via the API Server.	The Control Plane, over the cluster network.

HOW IT ALL WORKS

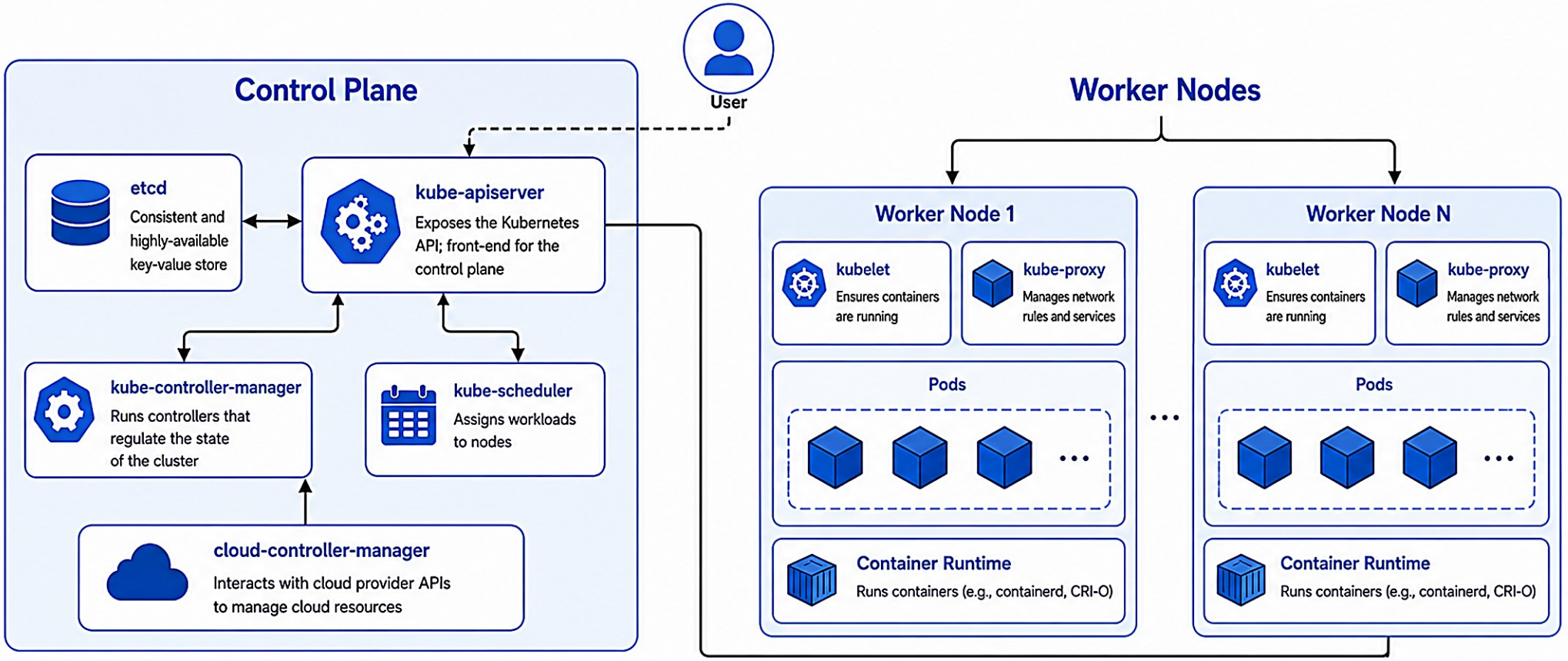
1. **Apply Config**
2. **API Server Validates**
3. **Scheduler Assigns Node**
4. **Kubelet & Kube-proxy Run It**
5. **App Available**

KEY POINTS

Declarative **Self-Healing** **Scalable** **Portable** **Extensible**

KEY TAKEAWAY Kubernetes follows a master-worker model: the control plane makes decisions, worker nodes run your containers -- all communication flows through the API Server.

Kubernetes Architecture Overview



Component Responsibilities

kube-apiserver
Provides the API interface for all cluster operations

etcd
Stores all cluster data in a consistent and reliable way

kube-controller-manager
Maintains the desired state of the cluster through controllers

kube-scheduler
Schedules pods to the most suitable nodes

cloud-controller-manager
Manages cloud resources and integrates with cloud providers

kubelet
Ensures containers in pods are running as expected

kube-proxy
Maintains network rules and enables service connectivity

Direct Communication — API Access — Component

CONTROL PLANE COMPONENTS

The Control Plane (Master) makes global decisions about the cluster and manages its state -- it is the brain of Kubernetes.

API Server

Central management point. Exposes the Kubernetes API; all communication goes through it.

etcd

Consistent, highly available key-value store -- the source of truth for the entire cluster state.

Scheduler

Watches for newly created Pods and selects the best node based on resources and policies.

Controller Manager

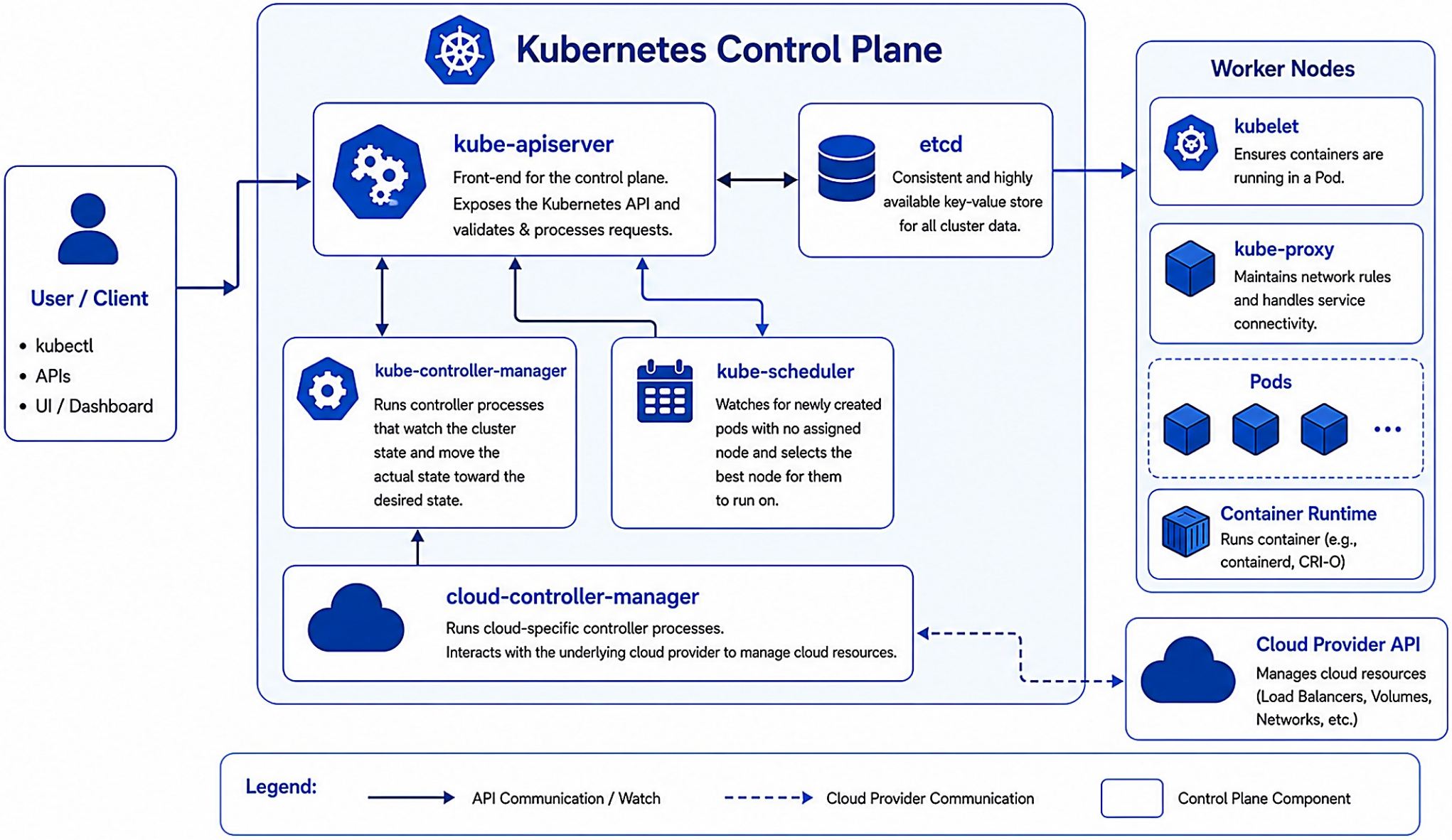
Runs controller processes that drive the current state toward the desired state (Node, ReplicaSet, ...).

KEY INTERACTIONS

- All communication with the cluster goes through the API Server -- it validates and processes every request.
- etcd stores all cluster data and configuration; every other component reads or writes through the API Server.
- The Scheduler assigns Pods to nodes; the Controller Manager continuously reconciles actual state toward desired state.

KEY TAKEAWAY The Control Plane is the brain of Kubernetes -- centralized management, a reliable state store, automated operations, and high availability by design.

Control Plane Components



WORKER NODES

Worker Nodes run your containerized applications -- they receive work from the Control Plane and execute Pods using available resources.

Run Pods

Host and run one or more Pods (containers) for your applications.

Provide Resources

Offer CPU, memory, storage, and network resources.

Execute Workloads

Pull container images, start containers, and keep them running.

Report Health

Continuously report node and Pod status back to the Control Plane.

HOW IT WORKS (STEP BY STEP)

1. **Scheduler Assigns Pod**
2. **Kubelet & Runtime Start It**
3. **Kube-proxy Configures Network**
4. **Health Reported**

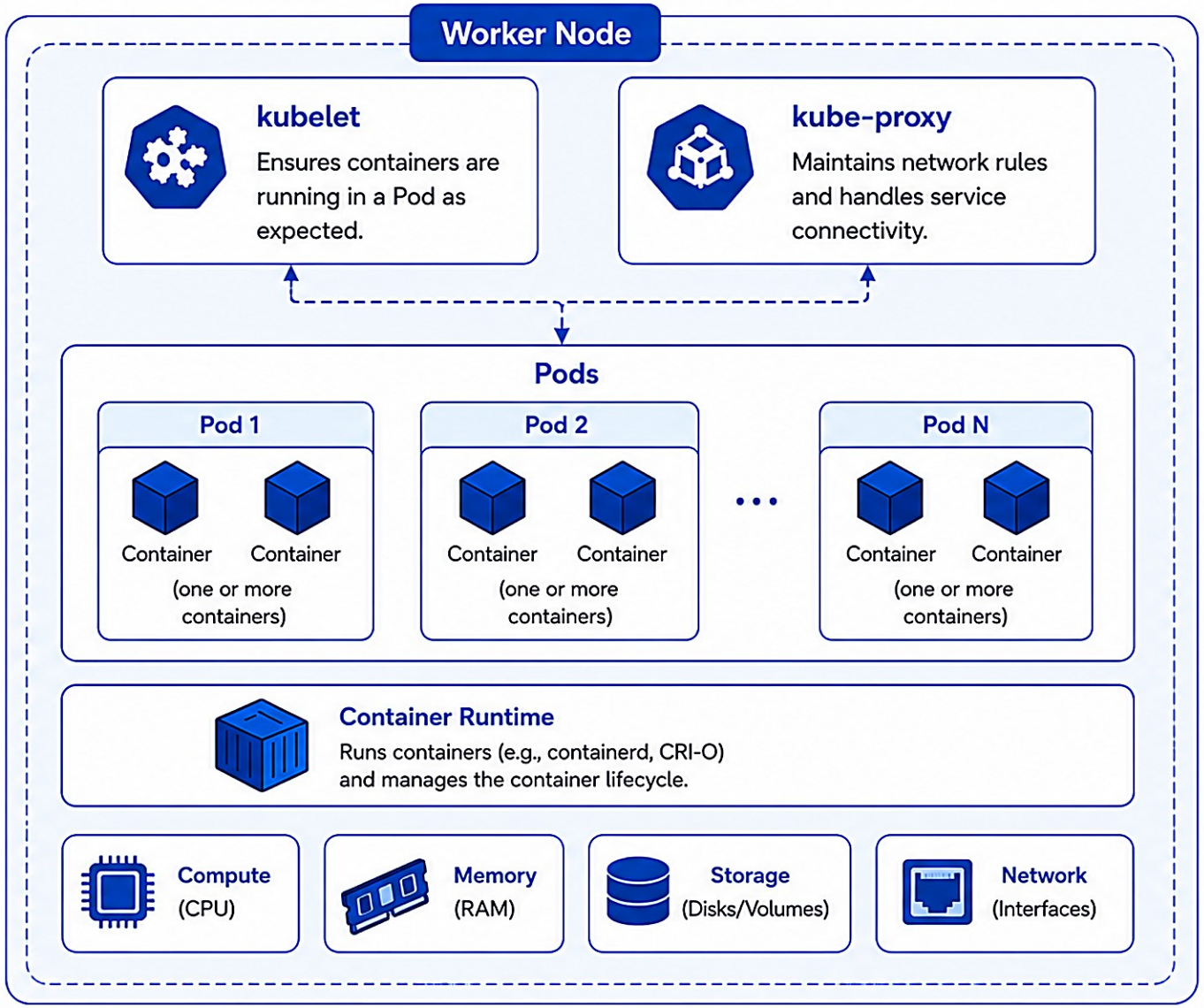
KEY POINTS

- Kubelet, kube-proxy, and the container runtime collaborate on every node to run and manage Pods.
- Self-healing: if a container or node fails, Kubernetes restarts it or reschedules the work elsewhere.

KEY TAKEAWAY Worker nodes are the machines in your cluster that run your containers and keep your applications available and healthy.

Worker Nodes

Worker nodes run your containerized applications and provide the compute, network, and storage resources.



Worker Node Responsibilities

Run Pods
Executes and manages application containers.

Health Monitoring
Monitors container and node health.

Networking
Provides networking connectivity for Pods and Services.

Storage
Mounts and manages volumes for Pods.

Resource Management
Utilizes node resources efficiently.

PODS

The smallest deployable unit in Kubernetes -- a Pod represents one or more containers that share the same network namespace and storage volumes.

KEY CHARACTERISTICS

One or More Containers — most commonly one container per Pod, sometimes tightly-coupled multiples.

Shared Network Namespace — all containers in a Pod share the same IP address and port space.

Shared Storage Volumes — containers can share volumes, making data accessible to all of them.

Co-located & Co-scheduled — all containers in a Pod run on the same node, together.

Ephemeral — Pods are transient; a failed Pod is replaced with a new one, not repaired.

HOW PODS ARE USED

Single Container Pod — runs one application container (e.g. crm-api). Most common.

Sidecar Pattern — a companion container for logging, monitoring, proxy, or security.

Adapter Pattern — helps the main container with data or format transformation.

Init Containers — run setup tasks before app containers start (e.g. wait for DB).

POD LIFECYCLE

KEY TAKEAWAY A Pod is like a 'logical host' for your containers -- keep containers in the same Pod only if they truly need to work closely together.

Pods

A Pod is the smallest deployable unit in Kubernetes. It represents a single instance of a running process in your cluster and can contain one or more containers that share storage, network, and configuration.

Key Characteristics

Smallest Deployable Unit
The basic building block in Kubernetes.

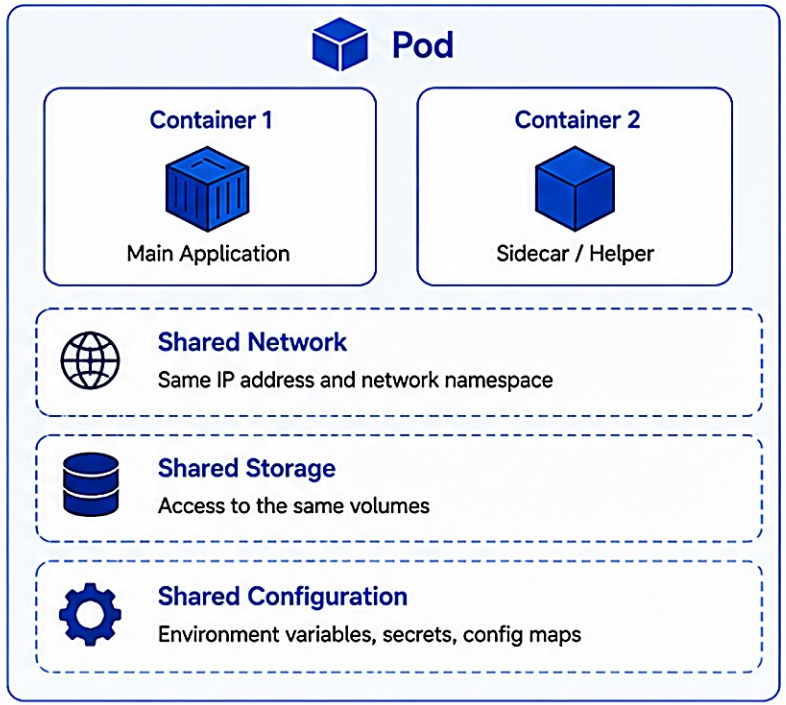
One or More Containers
Tightly coupled containers that work together.

Shared Resources
Containers in a Pod share the same network namespace, storage volumes, and configurations.

Ephemeral
Pods are created, scheduled, and destroyed. They are not self-healing by themselves.

Managed by Controllers
Deployments, StatefulSets, DaemonSets, and Jobs manage Pods.

Pod Architecture



Pod Lifecycle

Pending
Pod is created but not yet scheduled.

Scheduled
Pod is assigned to a node.

Running
Containers are running successfully.

Succeeded / Failed
Pod has completed or containers have failed.

Terminated
Pod is removed from the cluster.

Pod Usage Examples

Web Application
Web server container with a log sidecar container.

Data Processing
Main processing container with a helper container for data retrieval.

Monitoring
Exporter sidecar with a main application container.

Init + App Container
Init container runs setup tasks before the main container starts.

REPLICASETS

A ReplicaSet ensures the desired number of Pod replicas are running at any given time, replacing Pods that fail, are deleted, or become unhealthy.

KEY FEATURES

Maintains Desired Replicas — ensures the specified number of Pods are always running.

Self-Healing — automatically replaces Pods that fail or are terminated.

Selector-Based — uses a label selector to manage a specific set of Pods.

Foundation for Deployments — Deployments use ReplicaSets to manage updates and scaling.

HOW REPLICASETS WORK

ReplicaSet YAML Example

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: crm-api-rs
  labels: { app: crm-api }
spec:
  replicas: 3
  selector:
    matchLabels: { app: crm-api }
  template:
    metadata:
      labels: { app: crm-api }
    spec:
      containers:
        - name: crm-api
          image: crm-api:1.0.0
```

The label selector (app: crm-api) is how the ReplicaSet knows which Pods belong to it -- the exact mechanism the Lab 42 guide warns about: a selector/label mismatch yields a Service with no Endpoints.

KEY TAKEAWAY ReplicaSets make sure the right number of Pod replicas are running and replace any that fail. They are the building blocks behind Deployments.

ReplicaSets

A ReplicaSet ensures that a specified number of pod replicas are running at any given time.
It replaces any pod that fails or is deleted and ensures the desired state is maintained.

Key Characteristics

Desired Replicas
Ensures a specified number of pod replicas are running.

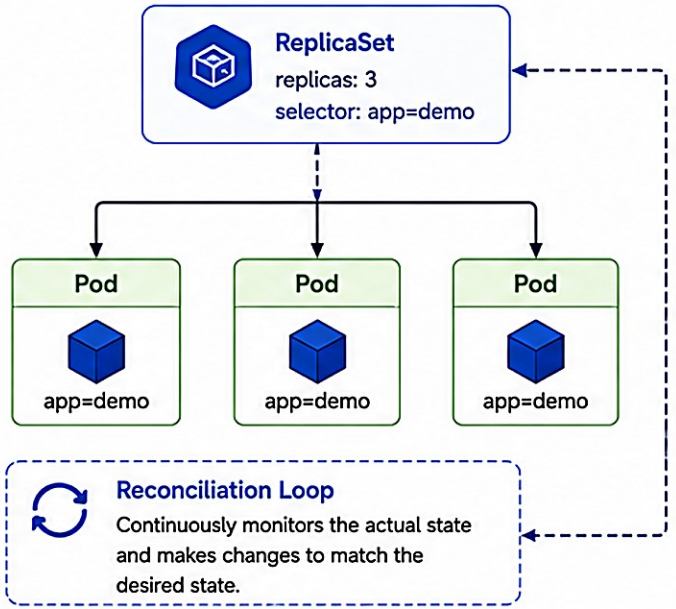
Self-Healing
Replaces failed, deleted, or unhealthy pods automatically.

Label Selector
Manages pods based on the labels you define.

Scalable
Easily scale up or down by updating the replica count.

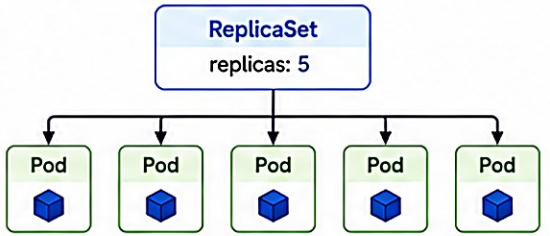
Stable
Maintains application availability and reliability.

How ReplicaSet Works

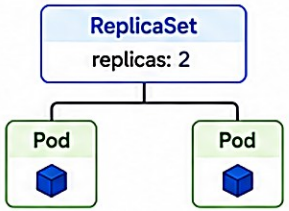


Scaling

Scale Up
Increase replica count



Scale Down
Decrease replica count



ReplicaSet vs. Other Controllers

ReplicaSet
Ensures a specified number of identical pods are running.

Deployment
Manages ReplicaSets and provides declarative updates and rollbacks.

StatefulSet
For stateful applications requiring stable network identity and storage.

Job / CronJob
For running tasks to completion, not for long-running services.

Example YAML

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: demo-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo
template:
  metadata:
    labels:
      app: demo
  spec:
    containers:
      - name: demo-container
        image: nginx:1.25
```

DEPLOYMENTS

A Deployment provides declarative updates for Pods and ReplicaSets -- describe the desired state, and Kubernetes changes reality to match it, safely.

KEY CAPABILITIES

Declarative Updates — you declare the desired state; Kubernetes handles the changes.

Rolling Updates — update your application with zero (or minimal) downtime.

Rollbacks — easily roll back to a previous stable revision if something goes wrong.

Scaling & Self-Healing — scale with a single command; automatically replaces failed Pods.

HOW DEPLOYMENTS WORK

Deployment YAML Example (crm-api)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: crm-api
  labels: { app: crm-api }
spec:
  replicas: 2
  selector:
    matchLabels: { app: crm-api }
  strategy:
    type: RollingUpdate
    rollingUpdate:
      { maxUnavailable: 0, maxSurge: 1 }
  template:
    metadata:
      labels: { app: crm-api }
    spec:
      containers:
        - name: crm-api
          image: crm-api:1.0.0
```

Lab 42's real Deployment sets replicas: 2 and prefers a digest-pinned image, not a moving :latest tag.

KEY TAKEAWAY Deployments provide a powerful and flexible way to manage your applications, enabling safe updates, rollbacks, scaling, and high availability with minimal effort.

Deployments

A Deployment provides declarative updates for Pods and ReplicaSets.
It manages the rollout and rollback of your application and ensures the desired state is maintained.

Key Features

Declarative Updates
You declare the desired state, Deployment handles the rest.

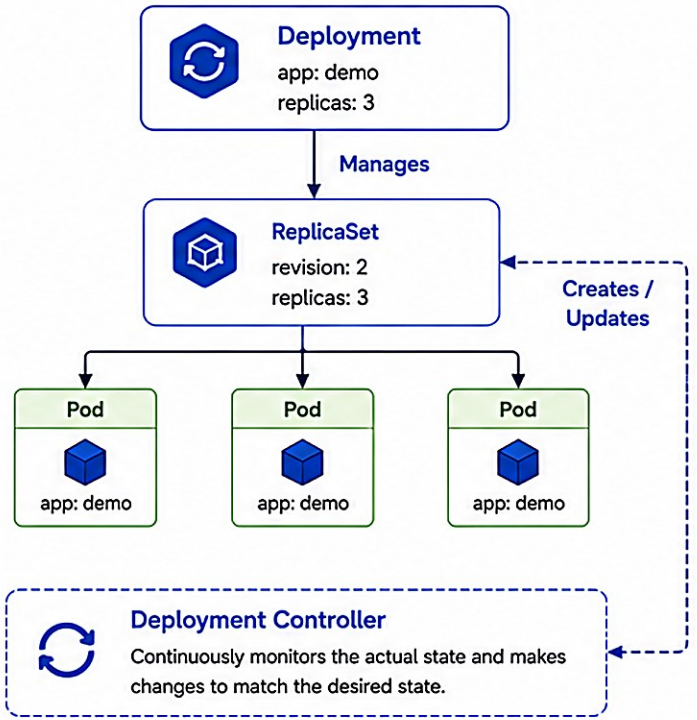
Rollouts & Rollbacks
Easily roll out new versions and roll back if needed.

Self-Healing
Automatically replaces failed or unhealthy Pods.

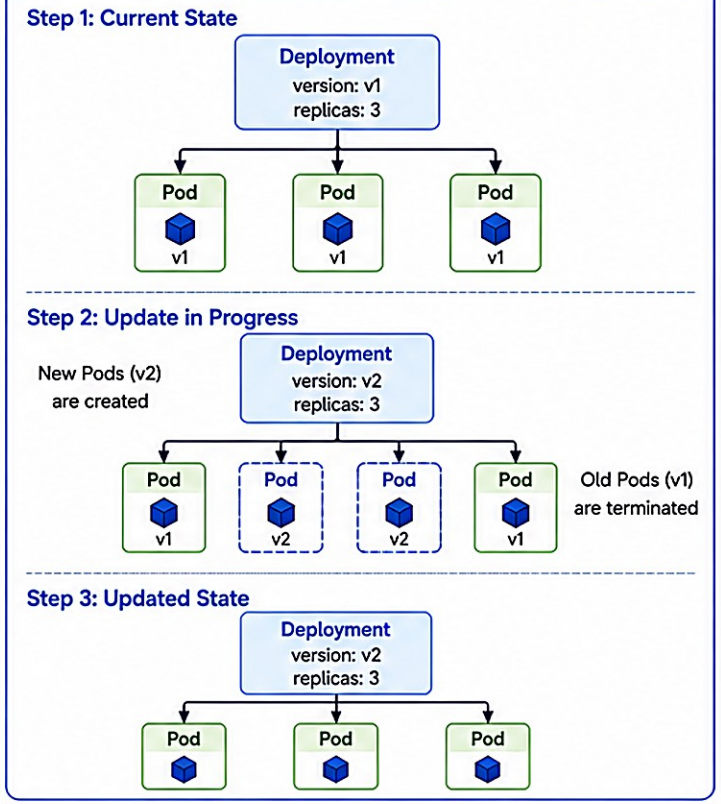
Scaling
Scale up or down the number of pod replicas.

Revision History
Keeps track of rollout history and revision changes.

How Deployments Work



Rolling Update Strategy



Deployment Operations

Create
Create a new Deployment.

Update
Update image, replicas, or configuration.

Rollback
Roll back to a previous revision.

Scale
Scale the number of pod replicas.

Delete
Delete a Deployment.

Example YAML

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: demo-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: demo
  template:
    metadata:
      labels:
        app: demo
    spec:
      containers:
      - name: demo-container
        image: nginx:1.25
```

TRY IT YOURSELF — EXERCISE 4: DEPLOYMENT YAML TODOS

Reinforces: the Deployment fields you just saw -- replicas, non-root security, digest-pinned image, and a readiness probe path.

FILL IN EVERY ____ (notes/lab42-yaml-todos.md)

deployment-skeleton.yaml (STARTER -- not a solution)

```
# deployment-skeleton.yaml
spec:
  replicas: _____
  template:
    spec:
      securityContext:
        runAsNonRoot: _____
        runAsUser: _____
      containers:
      - name: crm-api
        image: _____@sha256:_____
        ports:
        - containerPort: _____
        readinessProbe:
          httpGet:
            path: _____
            port: _____
```

#	What To Do
1	Fill replicas (2), non-root (true), a UID, and the container port (8080).
2	Fill the image digest placeholder -- never rely on :latest alone.
3	Fill the readiness probe path (e.g. /actuator/health/readiness) and port.
4	Add a resources.requests/limits block for CPU and memory.
5	Do NOT run kubectl apply -- this exercise stays on paper.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-yaml-todos.md (workspace: examples/module-42-exercises).

SERVICES

A Service provides a stable IP address and DNS name for a set of Pods and load balances traffic across them -- decoupling clients from Pod churn.

KEY FEATURES

Stable Endpoint — provides a stable IP and DNS name for your application.

Load Balancing — distributes traffic across healthy Pods backed by the Service.

Decoupling — clients don't need to know Pod IPs, which can change at any time.

Service Discovery — automatic DNS records for easy discovery inside the cluster.

TYPES OF SERVICES

Type	What It Does
ClusterIP (default)	Internal IP only -- accessible within the cluster. Lab 42 uses this for crm-api.
NodePort	Exposes the Service on each node's IP at a static port.
LoadBalancer	Provisions an external load balancer (cloud provider) to expose the Service.
ExternalName	Maps the Service to an external DNS name -- no proxying of traffic.

Lab 42 pairs a ClusterIP Service (internal only) with an Ingress at the edge -- the Service alone is never directly internet-reachable.

HOW SERVICES WORK

1. **Client Sends Request**
2. **Service Selects a Pod**
3. **Traffic Forwarded**
4. **Response Returned**

KEY TAKEAWAY Services provide stable networking, load balancing, and service discovery for your applications -- use the right Service type to expose your app as needed.

SERVICES

Services provide a stable network endpoint to access a set of Pods.

WHY SERVICES?

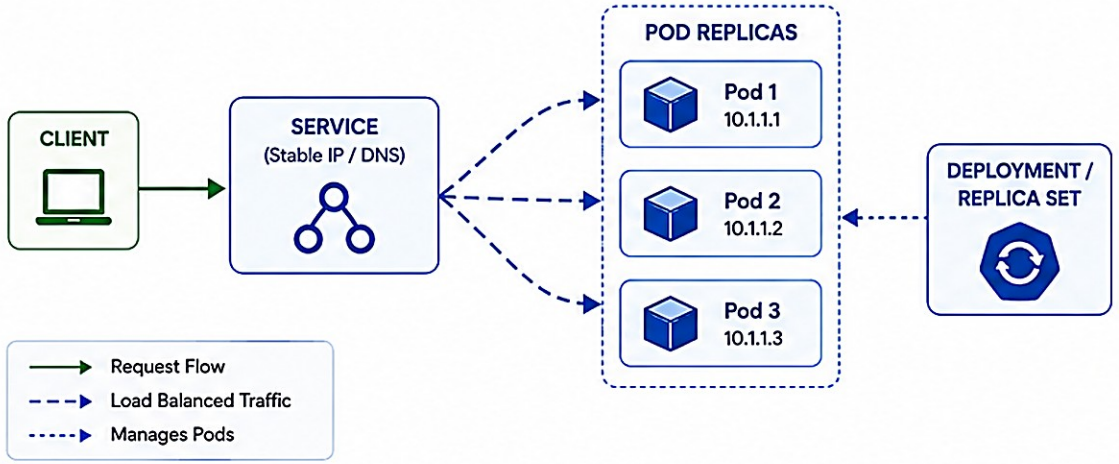
Stable Endpoint
Provides a consistent IP and DNS name.

Load Balancing
Distributes traffic across Pod replicas.

Decoupling
Pods are ephemeral, Services are stable.

Abstraction
Consumers don't need to know Pod IPs.

HOW SERVICES WORK



SERVICE OBJECT

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: ClusterIP
  selector:
    app:  my-app
  ports:
   - protocol: TCP
     port: 80
     targetPort: 8080
```

apiVersion, kind:	Define the object type.
metadata.name:	Name of the Service.
spec.type:	Type of Service (ClusterIP, NodePort, LoadBalancer, ExternalName).
spec.selector:	Selects Pods using labels.
spec.ports.port:	Port exposed by the Service.
spec.ports.targetPort:	Port on the Pod to forward traffic.

KEY FEATURES

Cluster IP
A virtual IP assigned within the cluster.

DNS Name
<service-name>.<namespace>.svc.cluster.local

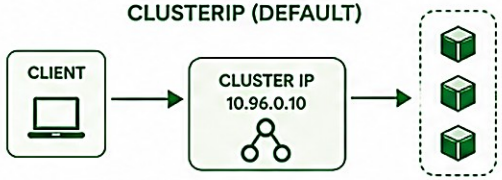
Load Balancing
Evenly distributes traffic to healthy Pods.

Health Aware
Routes traffic only to healthy Pods.

Port & Protocol
Supports TCP, UDP, and SCTP.

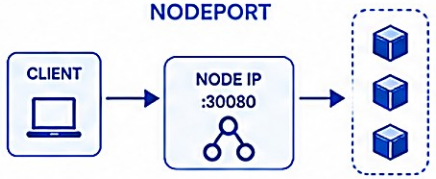
TYPES OF SERVICES

CLUSTERIP (DEFAULT)



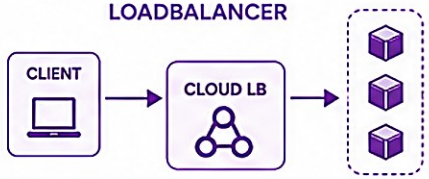
Exposes the Service on a cluster-internal IP. Accessible only within the cluster.

NODEPORT



Exposes the Service on each Node's IP at a static port (NodePort). Accessible from outside the cluster.

LOADBALANCER



Provisions an external Load Balancer that routes traffic to Nodes, then to Pods.

EXTERNALNAME

Maps the Service to an external DNS name. No proxying or load balancing.

CONFIGMAPS

ConfigMaps store non-sensitive configuration data in key-value pairs -- decoupling configuration from container images.

KEY FEATURES

Externalized Configuration — separate configuration from your application code and images.

Easy to Update — update configuration without rebuilding container images.

Multiple Consumption Options — use as environment variables, files, or arguments.

Not for Sensitive Data — never store passwords, tokens, or secrets here -- use Secrets.

CONSUMING IN A POD

As Environment Variables — envFrom.configMapRef injects every key as an env var.

As Files in a Volume — mounted at a path, one file per key.

crm-api-config (real Lab 42 ConfigMap)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: crm-api-config
data:
  SPRING_PROFILES_ACTIVE: "kubernetes"
  CRM_DB_URL: "jdbc:postgresql://..."
  CRM_DB_USERNAME: "crm_app"
```

Notice: the DB URL and username are here, but the DB password is NOT -- that goes in a Secret, next slide.

COMMON USE CASES

- Application Settings
- Feature Flags
- Runtime Config
- Microservices Config

KEY TAKEAWAY ConfigMaps help you manage configuration separately from your container images -- like Lab 42's crm-api-config for the Spring profile and DB URL. Never store passwords here.

ConfigMaps

ConfigMaps store non-confidential configuration data in key-value pairs.
Applications can consume this configuration as environment variables, files, or command-line arguments.

Key Characteristics

Decouple Configuration
Separate configuration from container image.

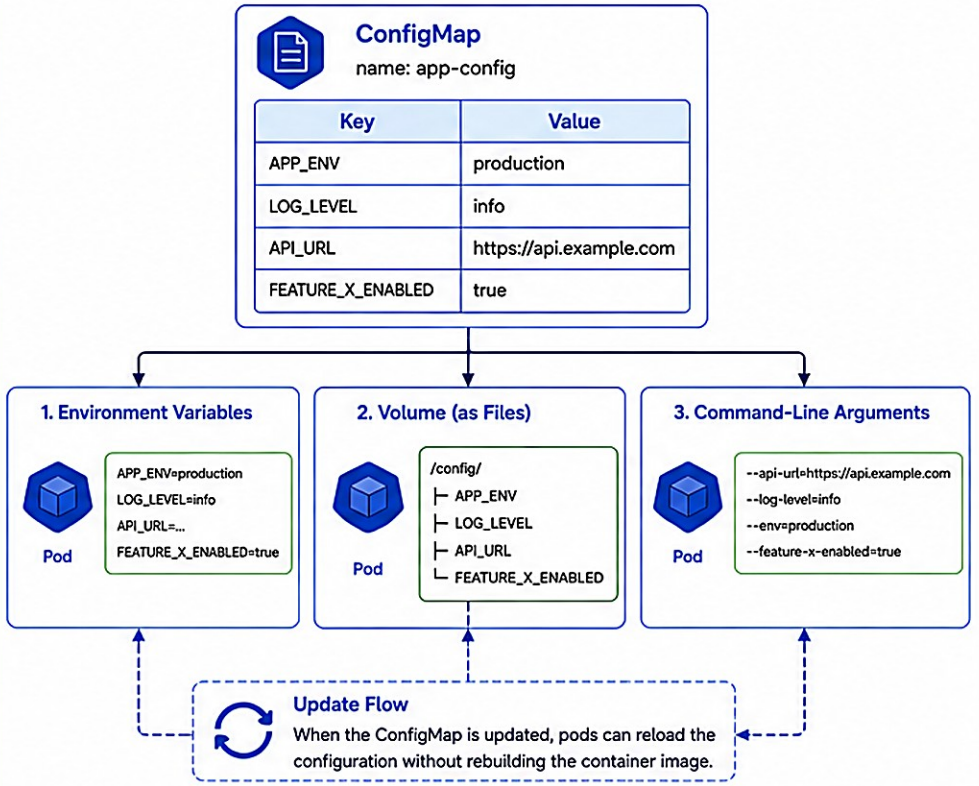
Key-Value Pairs
Store configuration as key-value pairs.

Multiple Ways to Use
Expose as environment variables, files in volumes, or CLI arguments.

Dynamic Updates
Update ConfigMap and applications can reload without rebuilding images.

Namespace Scoped
ConfigMaps are scoped to a namespace.

How ConfigMaps Work



Example YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  API_URL: "https://api.example.com"
  FEATURE_X_ENABLED: "true"
```

Ways to Use in a Pod

As Environment Variables

```yaml
env:
 - name: APP_ENV
   valueFrom:
    configMapKeyRef:
      name: app-config
      key: APP_ENV
```

As Volume (Files)

```yaml
volumes:
 - name: config-volume
   configMap:
     name: app-config
mounts:
 - name: config-volume
   mountPath: /config
```

As Command-Line Arguments

```yaml
args:
 - --api-url=$(API_URL)
 - --log-level=$(LOG_LEVEL)
```

Common Use Cases

Application Settings
Manage app behavior without code changes.

Feature Flags
Enable or disable features dynamically.

Environment Configuration
Store environment-specific values (dev, test, prod).

External API Endpoints
Manage URLs and endpoints centrally.

Logging Configuration
Adjust log levels and formats.

SECRETS

Secrets let you store and manage sensitive data such as passwords, tokens, and keys -- kept out of your container images and manifests.

KEY FEATURES

Secure Storage — sensitive data is stored in base64-encoded format.

Decoupled from Images — keeps secrets out of container images and code repositories.

Access Control — RBAC rules control who can create and read Secrets.

Encrypted at Rest — stored in etcd, encrypted at rest (when enabled).

TYPES OF SECRETS

- Opaque (default)
- kubernetes.io/tls
- dockerconfigjson
- ssh-auth

Real Lab 42 pattern -- create out-of-band

```
kubectl create secret generic crm-api-secrets \
  --from-literal=CRM_DB_PASSWORD='...' \
  --dry-run=client -o yaml   # review; apply in training ns only
```

Commit secret.example.yaml with keys listed and NO values. If a password is ever committed: remove it, rotate the training secret, and involve the instructor before rewriting history.

BEST PRACTICES

- Secrets only for sensitive data
- Restrict access via RBAC
- Rotate regularly

KEY TAKEAWAY Secrets store sensitive data like the CRM DB password -- created out-of-band in the cluster, never committed to Git. Commit only secret.example.yaml with no values.

Secrets

Secrets store sensitive information such as passwords, tokens, and keys.
They are encoded in Base64 and can be mounted as files or exposed as environment variables in Pods.

Key Characteristics

Store Sensitive Data
Designed to store sensitive information securely.

Base64 Encoded
Values are Base64 encoded (not encrypted by default).

Decouple from Images
Keep secrets out of container images and code.

Multiple Ways to Use
Use as environment variables or mounted files.

Update & Reload
Update secrets and reload pods without rebuilding images.

Namespace Scoped
Secrets are scoped to a namespace.

How Secrets Work

Secret
name: db-credentials

Key	Value (Base64 Encoded)
username	YWRtaW4=
password	cGFzc3dvcmQxMjM=
token	ZXlKaGJHY2lPaUpTVXpJMU5pSjku.Li4=

As Environment Variables

Pod

```
env:
- name: DB_USERNAME
  valueFrom:
    secretKeyRef:
      name: db-credentials
      key: username
```

As Files in a Volume

Pod

```
/etc/secret
  username
  password
  token
```

Pod Uses Secret
The application reads the secret from environment variables or files.

Example YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: default
type: Opaque
data:
  username: YWRtaW4=
  password: cGFzc3dvcmQxMjM=
  token: ZXlKaGJHY2lPaUpTVXpJMU5pSjku.Li4=
```

Ways to Use in a Pod

As Environment Variables

```yaml
env:
 - name: DB_PASSWORD
   valueFrom:
     secretKeyRef:
       name: db-credentials
       key: password
```

As Volume (Files)

```yaml
volumes:
 - name: secret-volume
   secret:
     secretName: db-credentials
volumeMounts:
 - name: secret-volume
   mountPath: /etc/secret
   readOnly: true
```

Common Use Cases

Database Credentials
Store usernames and passwords.

API Keys & Tokens
Store API keys, tokens, and access keys.

TLS Certificates
Store TLS certs, private keys, and CA bundles.

External Service Credentials
Store credentials for external services.

Application Secrets
Store app-specific secrets like encryption keys.

Note

- Secrets are Base64 encoded, not encrypted. Anyone with access to the cluster can decode them.
- Enable etcd encryption at rest and restrict RBAC permissions to protect secrets.

TRY IT YOURSELF — EXERCISE 1: MAP K3S MANIFESTS

Reinforces: naming every object needed to run crm-api on the cohort k3s cluster, before moving to the deployment workflow.

REFERENCE (notes/lab42-manifest-map.md)

Object	Holds	Must Not Hold
ConfigMap	Non-secret URLs/flags	DB passwords
Secret	Credentials (out-of-band)	Values in Git
Deployment	Pod template, probes	HostPath secrets
Service	ClusterIP ports	TLS private keys
Ingress	Host/path/TLS redirect	App business logic

STEPS

- List objects: Namespace (student), ConfigMap, Secret (ref only), Deployment, Service, Ingress (Traefik).
- Propose app labels: app=crm-api, lab=42, customer-fixture=synthetic.
- Note the image must be digest-pinned from Lab 41 -- not :latest alone.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 1 before continuing: exercises/exercise-01-manifest-map.md (workspace: examples/module-42-exercises).

TRY IT YOURSELF — EXERCISE 2: CONFIG VS SECRET

Reinforces: classifying real CRM settings into ConfigMap vs Secret before Lab 42 -- an analysis exercise.

SORT EACH SETTING (notes/lab42-config-vs-secret.md)

ConfigMap (non-secret)	Secret (sensitive)
Datasource URL host, Kafka bootstrap servers	Database password
JWT issuer URI	TLS private keys / certificates
Log level, feature flags	Registry pull credentials

STEPS

- Sort the full list: datasource URL host, DB password, Kafka bootstrap, JWT issuer URI, log level, feature flags.
- Confirm the reference pattern: Secret data is created out-of-band; Git only ever gets secret.example.yaml with no values.
- Confirm CUS-1001 / CUS-1002 are app fixtures, not Kubernetes config keys -- don't confuse the two.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-config-vs-secret.md (workspace: examples/module-42-exercises).

KUBERNETES DEPLOYMENT WORKFLOW

From manifest to running application -- a Deployment automates the rollout and ensures the desired state is achieved and maintained.

FROM MANIFEST TO RUNNING APPLICATION

WHAT'S HAPPENING BEHIND THE SCENES

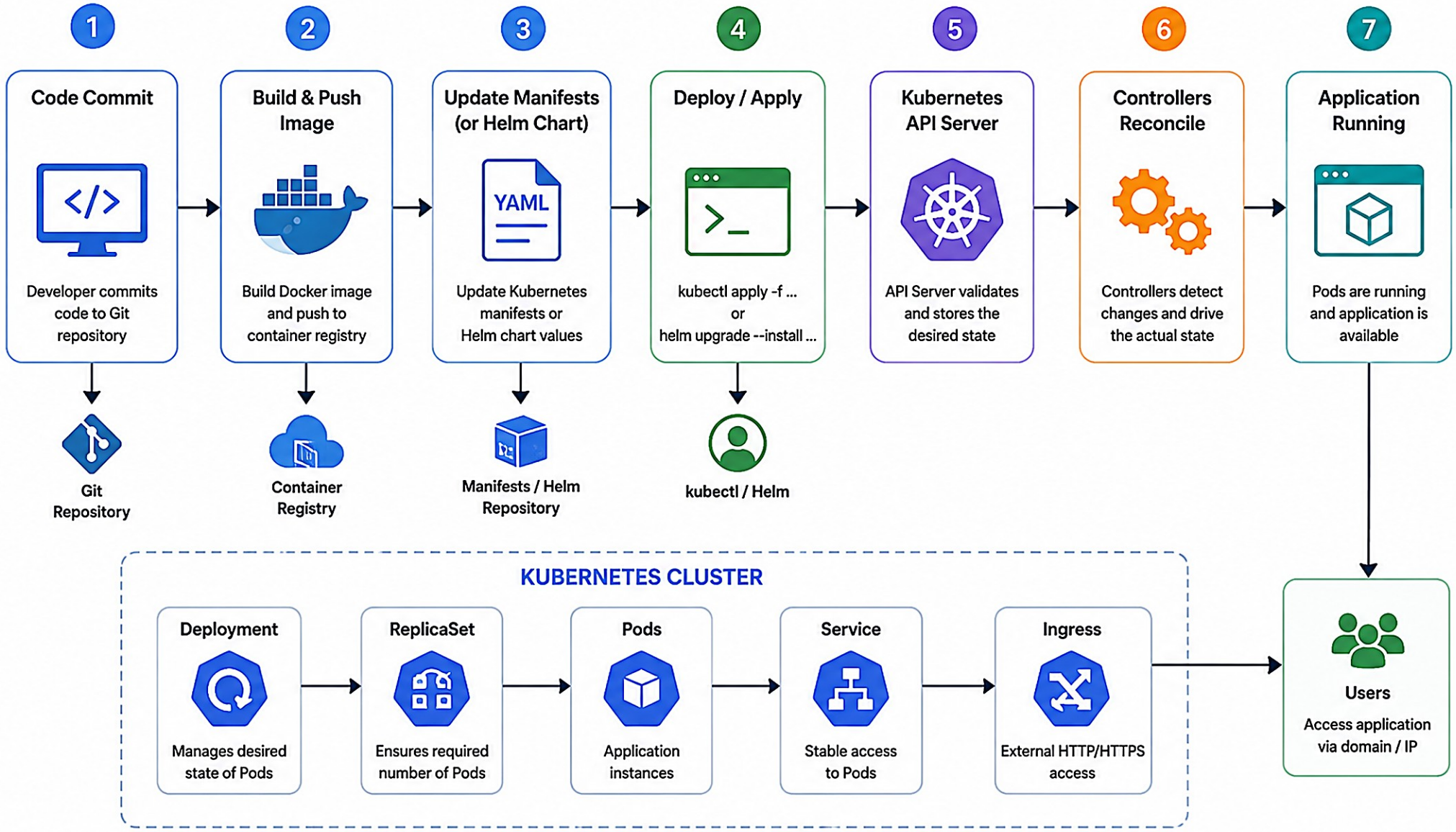
- API Server validates and stores the Deployment object; etcd becomes the source of truth.
- Deployment Controller continuously monitors the Deployment and ensures the desired state.
- ReplicaSet Controller ensures the correct number of Pods are running per the replica count.
- Kubelet runs containers in Pods on each worker node; health checks monitor Pod status.

USEFUL COMMANDS

Command	Does
kubectl apply -f deployment.yaml	Create/update
kubectl get deployments	List Deployments
kubectl get pods	List Pods
kubectl rollout status deploy/X	Check progress
kubectl rollout history deploy/X	View revisions
kubectl scale deploy/X --replicas=5	Scale

KEY TAKEAWAY Deployments provide a powerful and reliable way to manage application updates -- automating rollouts, scaling, and recovery so your application always runs the desired version.

Kubernetes Deployment Workflow



ROLLING UPDATES

Rolling Updates gradually replace old Pods with new ones in a controlled way -- Kubernetes ensures your application stays available and healthy throughout.

KEY BENEFITS

Zero/Minimal Downtime — new Pods are ready before old Pods are terminated.

Safe & Controlled — control exactly how many Pods are updated at a time.

Quick Rollback — easily revert to the previous stable version if issues occur.

Automated — Kubernetes Deployments handle the entire process for you.

Rolling Update Strategy (YAML)

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 25%   # max Pods unavailable
      maxSurge: 25%         # max extra Pods created
```

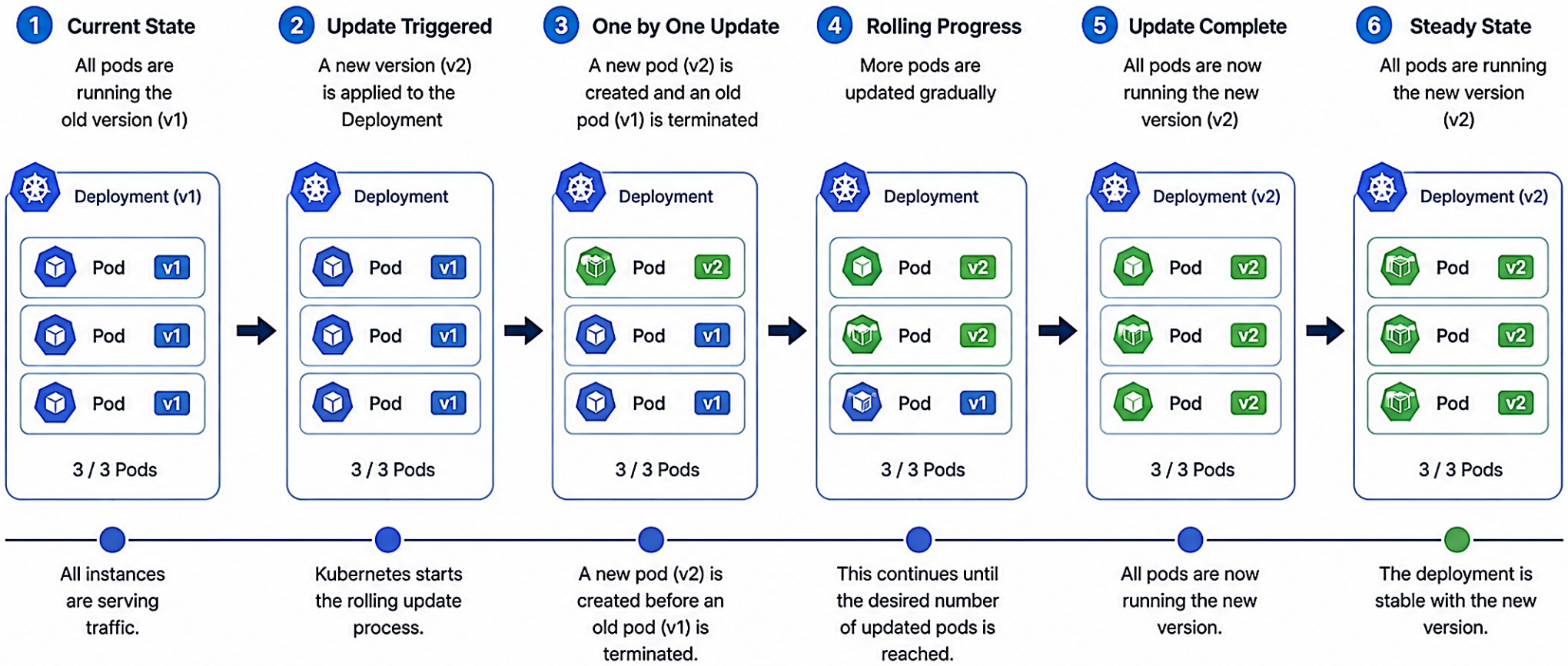
At every step: a new Pod becomes Ready, traffic shifts to it, and only then is an old Pod terminated -- this is exactly why readiness probes (covered on Self-Healing Applications) must be accurate: a false-positive Ready would route traffic to a Pod that isn't actually able to serve it.

UPDATE STRATEGY PARAMETERS

maxUnavailable	maxSurge	Meaning
0	1	Most safe -- no downtime (Lab 42's own setting)
25%	25%	Balanced (Kubernetes default)
1	0	One-by-one replacement

KEY TAKEAWAY Rolling Updates ensure your application is updated safely without downtime by gradually replacing old Pods with new ones -- with a quick rollback always available.

Rolling Updates



Pod (v1) Old Version

Pod (v2) New Version

Transition

3 / 3 Ready / Desired Pod Count

Benefits

- Zero downtime
- Gradual rollout reduces risk
- Automatic rollback on failure

ROLLBACKS

Rollbacks let you quickly revert to a previous stable version and restore your application to a known-good state with minimal downtime.

KEY BENEFITS

Quick Recovery — revert to a previous stable version in seconds.

Version History — the Deployment keeps a history of all revisions.

Safe & Controlled — traffic is shifted back gradually using the rollout strategy.

REVISION HISTORY EXAMPLE

Rev.	Change-Cause	Image	Status
3	Update to v2 (failed)	crm-api:v2	Failed
2	Update to v1.1	crm-api:v1.1	Success
1	Initial deployment	crm-api:v1	Success

Common Rollback Commands

```
kubectl rollout history deployment/crm-api
kubectl rollout undo deployment/crm-api
kubectl rollout undo deployment/crm-api --to-revision=2
```

Important: rollback only works if the previous ReplicaSet is still available -- Kubernetes scales it back up while scaling the bad one down.

This mirrors Lab 42's Step 9 rollout drill exactly: deploy a deliberately bad revision in a sandbox, observe the failed rollout, then run rollout undo and re-verify that the CUS-1001 smoke test and correlation ID lab-request-001 still work on the restored revision.

KEY TAKEAWAY Rollbacks provide a safety net to quickly recover from bad deployments. Use rollout history and built-in commands to restore stability fast.

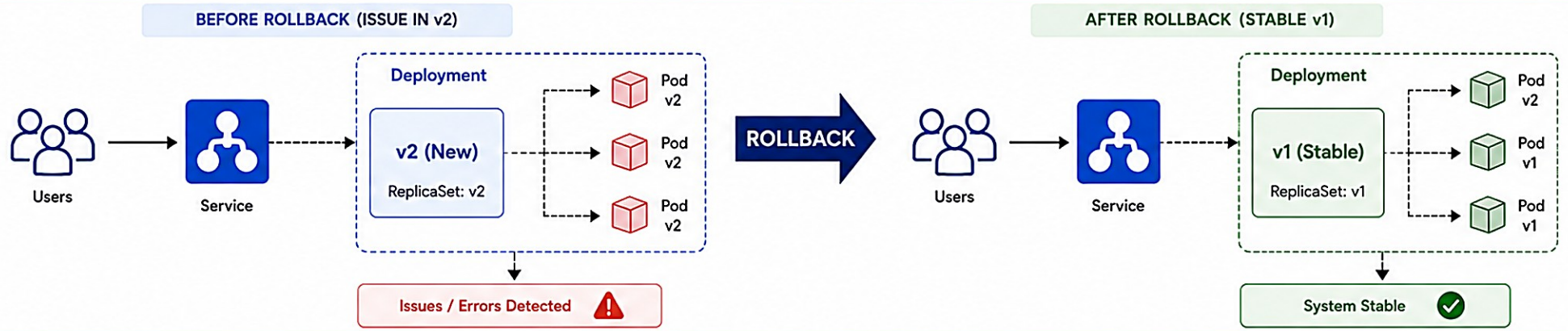
ROLLBACKS

Revert to a previous stable version when an issue is detected.

ROLLBACK WORKFLOW

1. **Issue Detected** — Monitor/alerts detect problems in the new version.
2. **Assess Impact** — Confirm the issue and decide to rollback.
3. **Initiate Rollback** — Trigger rollback to the previous stable revision.
4. **Kubernetes Rolls Back** — Kubernetes updates the ReplicaSet to the previous version.
5. **Traffic Restored** — Traffic is routed to the stable version.
6. **Verify & Monitor** — Verify the system is healthy and monitor closely.

HOW ROLLBACK WORKS (EXAMPLE)



ROLLBACK COMMANDS

```
kubectl rollout undo deployment/<name>
```
Rollback to the previous revision.

```
kubectl rollout undo deployment/<name> --to-revision=N
```
Rollback to a specific revision.

```
kubectl rollout history deployment/<name>
```
View revision history.

REVISION HISTORY (EXAMPLE)

REVISION	CHANGE	IMAGE	STATUS	TIME
3	Rollback to v1	app:v1	Deployed	10:30 AM
2	Update to v2	app:v2	Failed	10:20 AM
1	Initial deploy v1	app:v1	Deployed	10:00 AM

BEST PRACTICES

- Monitor continuously and set up alerts.
- Test updates in staging before production.
- Keep revision history for quick recovery.
- Automate rollback triggers for critical issues.

TRY IT YOURSELF — EXERCISE 5: ROLLOUT & ROLLBACK

Reinforces: rollout success/rollback commands and evidence, right after Rolling Updates and Rollbacks.

STEPS (notes/lab42-rollout-rollback.md)

Step	What To Do
1 — Rollout Watch	List the checks: kubectl rollout status, pod Ready, Ingress HTTP check, CRM get CUS-1001.
2 — Rollback Story	Describe rehearsing a bad revision, then rollout undo to the known-good digest.
3 — Evidence	Name screenshot folders under notes/screenshots/lab-42/ for before/after.
4 — Correlation	Include header lab-request-001 on every smoke call in the checklist.

Checklist commands to include

```
kubectl rollout status deployment/crm-api --timeout=180s
kubectl rollout history deployment/crm-api
kubectl rollout undo deployment/crm-api
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-rollout-rollback.md (workspace: examples/module-42-exercises).

SCALING APPLICATIONS

Kubernetes makes it easy to scale your applications manually or automatically to ensure performance, availability, and cost efficiency.

KEY BENEFITS

High Availability — handle more traffic and keep your application available.

Cost Efficiency — scale down when demand decreases to save resources.

Elasticity — scale up/down automatically based on real-time metrics.

SCALING STRATEGIES

Strategy	What It Does
Vertical	Increase CPU/Memory of existing Pods (requires a restart).
Horizontal (HPA)	Increase/decrease the number of Pods -- no downtime, ideal for stateless apps.
Cluster	Add more nodes to the cluster to support more Pods overall.

Manual scaling (Lab 42 Step 10)

```
kubectl scale deployment/crm-api --replicas=2
kubectl get pods -l app=crm-api -o wide
```

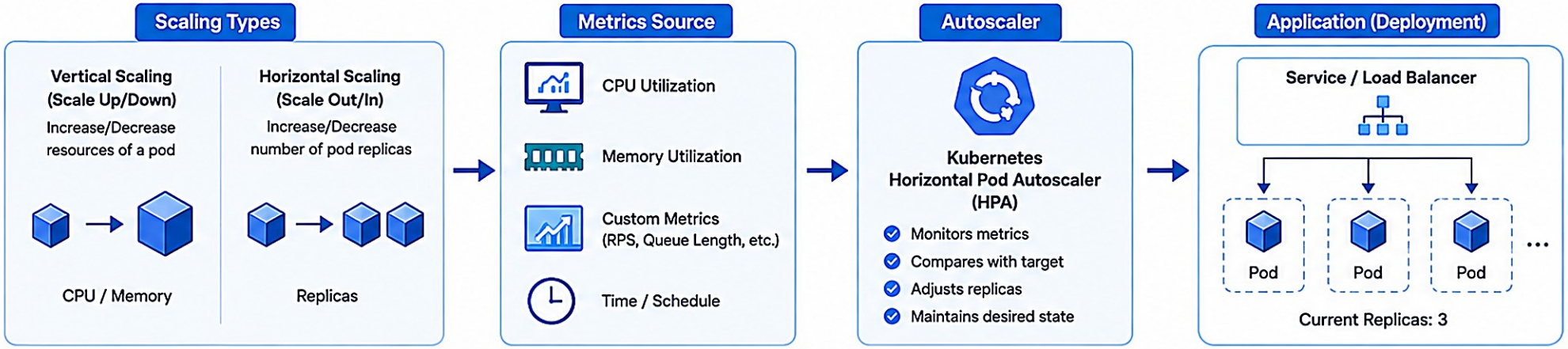
HorizontalPodAutoscaler (HPA) YAML

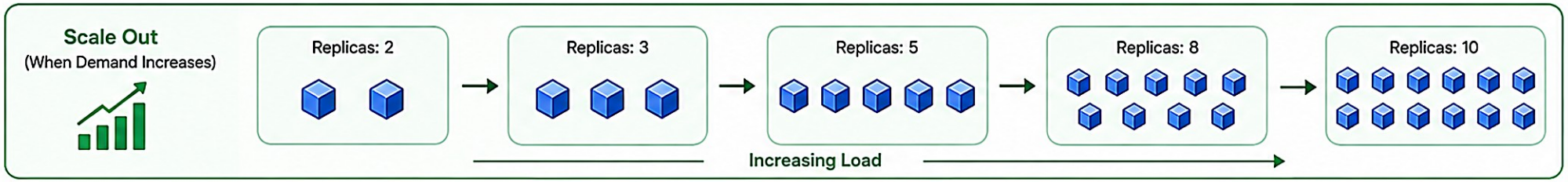
```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: crm-api-hpa }
spec:
  scaleTargetRef: { kind: Deployment,
    name: crm-api }
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource: { name: cpu }
      target: { type: Utilization,
        averageUtilization: 70 }
```

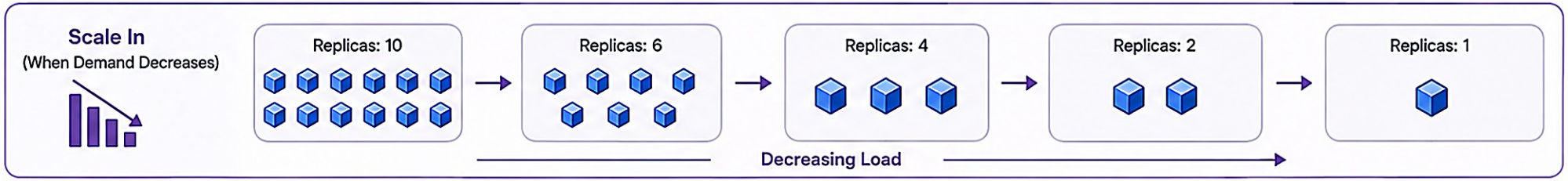
KEY TAKEAWAY Kubernetes provides powerful scaling capabilities to keep applications responsive, available, and efficient -- manual scaling for immediate needs, HPA for intelligent automatic scaling.

Scaling Applications

Automatically adjust application resources to meet demand







WHAT IS K3S?

k3s is a lightweight, certified Kubernetes distribution designed for easy installation, low resource usage, and edge/IoT/CI environments.

KEY FEATURES

Lightweight — a small binary (~100 MB) with a low memory and CPU footprint.

Easy to Install — single-binary install; works on Linux, Raspberry Pi, ARM, VM, cloud.

Embedded Components — uses SQLite by default -- no external etcd required for single-node.

Built-in Add-ons — Traefik Ingress, CoreDNS, metrics-server, and ServiceLB included.

CNCF Certified — a certified Kubernetes distribution, secure by default.

K3S VS KUBERNETES (UPSTREAM)

Feature	k3s	Kubernetes (Upstream)
Installation	Single binary	Multiple binaries / complex setup
Footprint	~100 MB, low memory	Higher memory and disk usage
Datastore	SQLite (default)	etcd (external)
Add-ons	Traefik, CoreDNS, metrics built in	Installed separately
Best For	Edge, IoT, CI/CD, small clusters	Enterprise, large-scale clusters

Quick Install (Single Node)

```
curl -sfL https://get.k3s.io | sh -
sudo systemctl status k3s
kubectl get nodes
```

This cohort's shared training cluster runs k3s -- lightweight enough for a classroom, with Traefik already built in for Lab 42's Ingress.

KEY TAKEAWAY k3s brings the power of Kubernetes to any environment -- lightweight, easy to install, secure by default, and perfect for edge, IoT, and this cohort's training cluster.

What is k3s?

k3s is a lightweight, certified Kubernetes distribution designed for edge, IoT, and single-node environments.

Lightweight
Small binary size (~60 MB) with minimal resource requirements.

Easy to Install
Single binary with zero external dependencies.

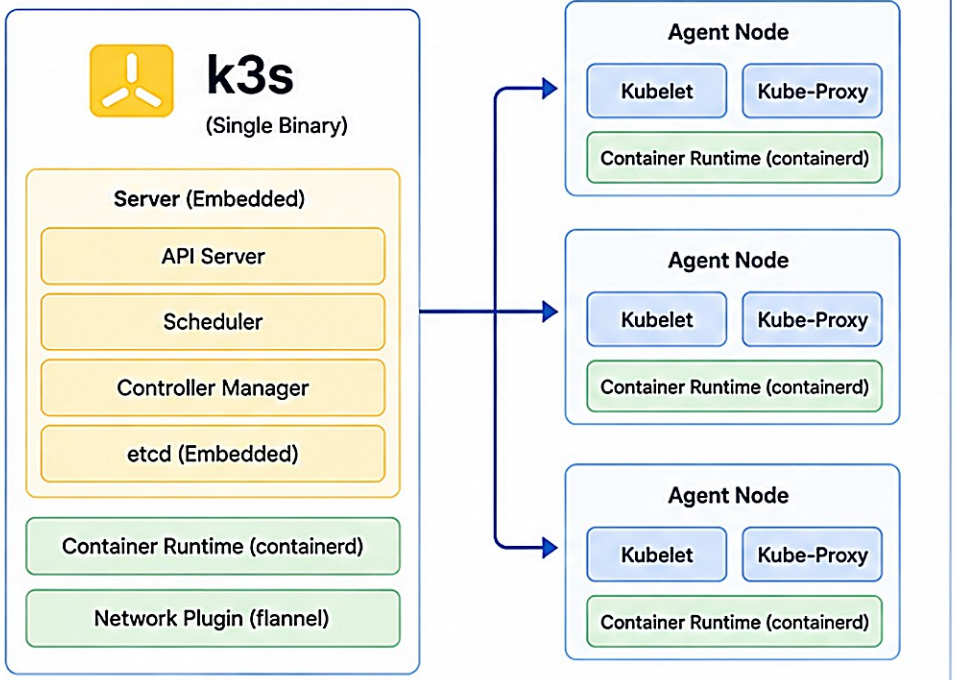
Certified Kubernetes
Conformance certified by the CNCF.

Edge & IoT Ready
Built for unreliable networks and resource-constrained environments.

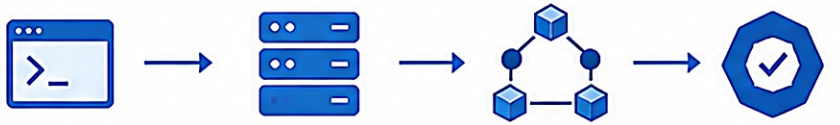
Secure by Default
RBAC enabled, encrypted communication, and security hardening.

Production Ready
Used in production by SUSE and thousands of organizations.

k3s Architecture



How k3s Works



1. Install k3s
Single command installation.

2. k3s Starts
All core Kubernetes components start as embedded services.

3. Add Agents
Add worker nodes to the cluster using a token.

4. Ready to Use
A lightweight Kubernetes cluster is ready.

Use Cases

- Edge Computing
- IoT / IIoT
- Remote Locations
- Development & Testing
- CI/CD & GitOps

Ideal For

- Low CPU / Memory Environments
- Intermittent / Unreliable Connectivity
- Single Node Deployments
- Secure Edge Deployments

K3S VS OPENSHIFT

Both are Kubernetes-based platforms, but built for different goals and environments -- simplicity and edge footprint vs enterprise security and scale.

Feature	k3s	OpenShift
Purpose	Lightweight Kubernetes for simplicity and efficiency.	Enterprise Kubernetes for security and scale.
Installation	Single binary, one command.	More complex, multiple components.
Ingress	Traefik (built in).	OpenShift Router (HAProxy) via Routes.
Security	Basic Kubernetes security (RBAC).	Enhanced: SCCs, OAuth, policies, compliance.
Operators	Not included (use Helm).	Operator Lifecycle Manager (OLM) built in.
Monitoring/Logging	Optional, install via Helm.	Built-in (Prometheus, Grafana, Loki).
Typical Use	Edge, IoT, CI/CD, small clusters.	Enterprise apps, regulated environments, large-scale.

WHEN TO CHOOSE?

k3s: simplicity, low resources, quick clusters

OpenShift: enterprise security, compliance, scale

KEY TAKEAWAY Both platforms give you the same core Kubernetes concepts from this module -- choose k3s for simplicity and low resource use, OpenShift for enterprise security, compliance, and scale.

k3s vs OpenShift

k3s		OpenShift
Purpose Lightweight Kubernetes distribution for edge, IoT, and single-node environments	Purpose	**Purpose** Enterprise Kubernetes platform for building, deploying, and managing applications at scale
Ease of Use / Installation Very easy to install and operate. Single binary, minimal dependencies.	Ease of Use / Installation	**Ease of Use / Installation** More complex installation and setup. Built for enterprise operations.
Footprint / Resources Very lightweight (~60 MB binary). Low CPU and memory usage.	Footprint / Resources	**Footprint / Resources** Heavier footprint. Requires more CPU, memory, and storage.
Features Core Kubernetes features only. Focus on simplicity and speed.	Features	**Features** Rich enterprise features: DevOps, CI/CD, Security, Monitoring, Logging, Service Mesh, and more.
Management Simple to manage. Best for small teams and edge use.	Management	**Management** Designed for enterprise teams. Role-based access, multi-tenancy, and centralized control.
Security Secure by default. Basic security controls.	Security	**Security** Advanced security: RBAC, SCC, Network Policies, Image Security, Compliance.
Use Cases Edge computing, IoT, CI/CD runners, development, and small workloads.	Use Cases	**Use Cases** Enterprise applications, microservices, hybrid cloud, regulated environments.
Scale Best for small to medium clusters (up to hundreds of nodes).	Scale	**Scale** Built for large-scale, critical workloads (thousands of nodes).
Support Community supported (Rancher Labs). SUSE offers support options.	Support	**Support** Red Hat supported. Enterprise-grade support and SLAs.
Cost Free and open source. No license fees.	Cost	**Cost** Subscription-based. Commercial support and add-ons.

OPENSHIFT ARCHITECTURE COMPARISON

OpenShift can be deployed in different architectures to fit your environment, scale, and operational needs.

Aspect	Local (Single Node)	Virtualization / IaaS	Public Cloud
Nodes	1 (all-in-one)	3+ control plane, 2+ worker	3+ control plane, autoscaling workers
High Availability	Not HA	Yes (3 or 5 control plane nodes)	Yes (auto-scale workers)
Installation	Very easy (one command)	Moderate (install & configure infra)	Easier (cloud integrations)
Infra Management	None (local machine)	You manage VMs, storage, network	Cloud provider manages infra services
Typical Use	Dev, learning, POCs, demos	Production, regulated, stable workloads	Elastic apps, global scale, rapid growth

WHICH ARCHITECTURE SHOULD YOU CHOOSE?

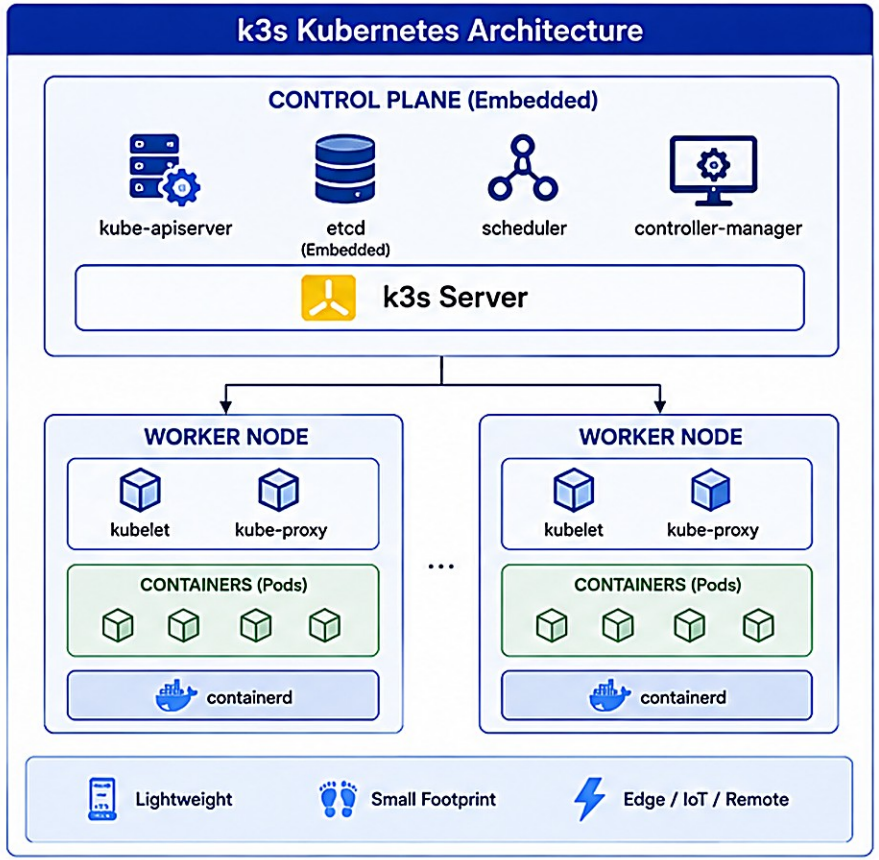
OpenShift Local — choose for local development, testing, and learning.

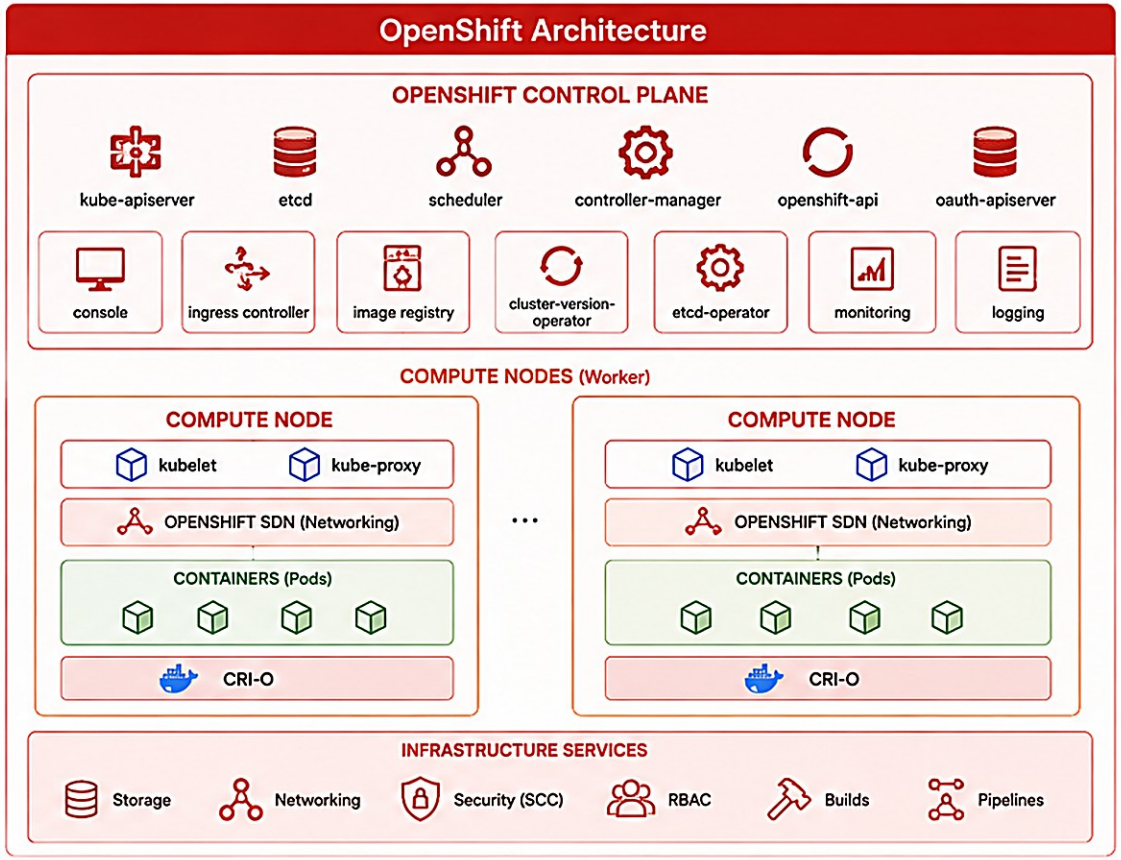
Virtualization / IaaS — choose for production workloads with full control and compliance needs.

Public Cloud — choose for agility, global scale, and leveraging managed cloud services.

KEY TAKEAWAY OpenShift provides the same enterprise Kubernetes platform across different architectures -- choose the deployment model that best fits your operational model, control, and scale requirements.

k3s vs OpenShift Architecture Comparison





FEATURE	k3s	ASPECT	OpenShift
Architecture	Single binary, all-in-one	**Design**	Modular, enterprise components
Control Plane	Embedded and lightweight	**Control Plane**	Separate, highly available
Container Runtime	containerd	**Runtime**	CRI-O
Networking	Flannel / CoreDNS	**Networking**	OpenShift SDN / OVN-Kubernetes
Security	Kubernetes RBAC	**Security**	SCC, RBAC, OAuth, Image Security
Registry	Option: local or external	**Image Registry**	Integrated internal registry
Use Cases	Edge, IoT, Dev/Test, Small clusters	**Use Cases**	Enterprise, Production, Multi-tenant
Management	Minimal, CLI (kubectl)	**Management**	Web Console, CLI, Operators

PROJECTS IN OPENSHIFT

Logical boundaries for teams, applications, and resources -- similar to Kubernetes namespaces, with added multi-tenancy and governance features.

KEY BENEFITS

Isolation — isolates resources, users, and policies from other teams and apps.

Multi-tenancy — safely share the same cluster across multiple teams.

Resource Management — apply quotas, limits, and policies per project.

Access Control — control who can access a project and what they can do.

EXAMPLE PROJECT USAGE

Project	Purpose	Placement
dev-team	Deployment, development	Non-production
qa-team	Testing & QA	Non-production
prod-team	Production, monitoring	Production

OpenShift Projects, k3s namespaces, and Lab 42's own per-student namespace all solve the same problem: isolating one team's or one student's resources from everyone else's on a shared cluster.

Project CLI Examples (oc)

```
oc get projects            # List all projects
oc new-project dev-team    # Create a new project
oc project dev-team        # Switch to a project
oc get all                 # List resources in it
```

PROJECT POLICIES

Resource Quotas **LimitRanges** **RBAC**

KEY TAKEAWAY Projects in OpenShift help you organize, secure, and manage resources efficiently in a shared cluster -- isolation, access control, and governance for teams and applications.

PROJECTS

A Project is a collection of related modules, resources, and configurations to build, run, and deploy an application.

MODULES

Logical units of functionality within a project.

- module-a
- module-b
- module-c
- module-n

RESOURCES

Files and assets used by the project.

- Images
- Properties
- Templates
- Data Files

CONFIGURATION

Settings and configurations that control the project behavior.

- Application Config
- Environment Config
- Security Config
- Build Config

BUILD & RUN

Commands and tools used to build, run, and test the project.

- Build
- Run
- Test
- Debug

OUTPUT

Artifacts generated after building the project.

- JAR / WAR
- Docker Image
- Reports
- Deployed App

PURPOSE

Organizes code, resources, and configurations to deliver a maintainable and scalable application.

ROUTES IN OPENSHIFT

Routes expose Services outside the cluster using hostnames, paths, and TLS termination -- OpenShift's counterpart to a Kubernetes Ingress.

KEY BENEFITS

External Access — expose applications to users outside the cluster.

TLS Termination — offload TLS at the edge for secure access.

Host & Path Based — route to multiple services using hostnames and paths.

Secure by Default — integrates with RBAC, network policies, and cert management.

ROUTE TYPES

Edge (default) **Re-encrypt** **Passthrough**

Example Route YAML (edge TLS)

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: crm-api
spec:
  host: crm-api.example.local
  to:
    kind: Service
    name: crm-api
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

Lab 42 exposes crm-api with a Traefik Ingress on k3s -- on OpenShift, this exact same job is done by a Route like the one above.

KEY TAKEAWAY Routes provide a simple, secure way to expose applications in OpenShift -- using the Ingress Controller to route external traffic to internal services via hostnames, paths, and TLS termination.

ROUTES

Routes define how an application responds to a request at a specific URL endpoint and HTTP method.

WHAT IS A ROUTE?

A route maps an incoming request to a specific handler function or controller.

ROUTE COMPONENTS

URL
The endpoint path (e.g., /users)

GET **HTTP METHOD**
The type of request (GET, POST, PUT, DELETE)

HANDLER
The function or controller that processes the request

RESPONSE
The data returned to the client

HOW ROUTES WORK

1. Client Request
   GET /users
2. Route Matching
   Find matching route
3. Handler Execution
   Process the request
4. Generate Response
   Send response to client

EXAMPLE ROUTES

```
GET     /users
  → getAllUsers

GET     /users/:id
  → getUserById

POST    /users
  → createUser

PUT     /users/:id
  → updateUser

DELETE  /users/:id
  → deleteUser
```

BEST PRACTICES

- Use meaningful and consistent URL patterns
- Use appropriate HTTP methods
- Keep routes focused and single-purpose
- Validate and sanitize route parameters
- Handle errors gracefully

PURPOSE

Routes provide a clear and organized way to handle client requests and build scalable web applications.

IMAGE STREAMS IN OPENSHIFT

Image Streams manage and track container images throughout their lifecycle -- versioning, traceability, and stable references for deployments.

KEY BENEFITS

Track Image History — maintains a history of all images with unique IDs (digests).

Stable References — tags (e.g. :latest, :v1) always point to a specific image.

Automatic Updates — tags can be updated automatically when new images are pushed.

Audit & Traceability — easily audit and roll back to previous image versions.

IMAGE STREAM STRUCTURE (EXAMPLE: crm-api)

Tag	Points To (Image ID)	Description
latest	sha256:abc123...	Most recent image
v1.1	sha256:def456...	Version 1.1 (Lab 41)
v1.0	sha256:789ghi...	Version 1.0 (initial)

This is exactly why every earlier slide's manifest recommended a digest-pinned image over a moving :latest tag -- an Image Stream tag CAN move to point at a new image, so :latest is not itself a stable reference; the underlying sha256 digest always is.

IMAGE STREAM LIFECYCLE

KEY TAKEAWAY Image Streams provide a powerful way to manage container images in OpenShift with versioning, traceability, and stable references -- enabling reliable deployments and easy rollbacks.

IMAGE STREAMS

Image Streams provide a way to track images over time. They act as an internal registry reference that keeps tags updated as new images are pushed.

1. IMAGE SOURCE

Images are built and pushed to an external registry.

External Registry

- myapp:1.0.0
- myapp:1.1.0
- myapp:1.2.0

2. IMAGE STREAM

An Image Stream tracks images from the registry and maintains tags.

Image Stream: myapp

latest	myapp:1.2.0
1.1	myapp:1.1.0
1.0	myapp:1.0.0

Image Stream stores metadata and history

3. AUTOMATIC UPDATES

When a new image is pushed to the registry, the Image Stream is updated.

New Image Pushed

myapp:1.3.0

Image Stream Updated

latest	myapp:1.3.0
1.1	myapp:1.1.0
1.0	myapp:1.0.0

4. CONSUMPTION

Applications and deployments reference the Image Stream tags, not image digests.

Deployment

Image: myapp:latest (Always pulls the latest image)

BENEFITS

- **Automatic Updates** — Keeps tags updated automatically
- **Consistent References** — Use stable tags like latest
- **History Tracking** — Tracks image history and changes
- **Decouples Consumers** — Applications don't need direct registry access

BUILD CONFIGURATIONS IN OPENSHIFT

A Build Configuration defines how source code is built into a container image -- source, strategy, and output image, all in one object.

BUILD STRATEGIES

Strategy	Use Case
Docker	Custom builds with complex, multi-stage Dockerfiles (Lab 41's own approach).
Source-to-Image (S2I)	Quick starts for standard languages (Java, Node.js, Go, ...).
Custom	Highly customized or legacy build requirements.

BUILD LIFECYCLE

1. **Source**
2. **Trigger**
3. **BuildConfig**
4. **Build Pod**
5. **Output Image**

Example BuildConfig (Docker Strategy)

```
apiVersion: build.openshift.io/v1
kind: BuildConfig
metadata:
  name: crm-api-build
spec:
  source:
    git: { uri: https://github.com/
      example/crm-api.git }
  strategy:
    type: Docker
  output:
    to: { kind: ImageStreamTag,
      name: "crm-api:latest" }
```

BUILD TRIGGERS

- **Manual**
- **Config Change**
- **Webhook (Git push)**
- **Image Change**

The Docker strategy builds match Lab 41's own multi-stage Dockerfile approach for crm-api -- the same build, running inside the cluster instead of a developer's laptop.

KEY TAKEAWAY Build Configurations automate and standardize turning source code into container images in OpenShift -- integrating with Image Streams, triggers, and pipelines for efficient, repeatable delivery.

BUILD CONFIGURATIONS

Build Configurations (BuildConfigs) define how to build an application. They specify the source, strategy, triggers, environment, and outputs.

COMPONENTS OF A BUILD CONFIGURATION

1. SOURCE

Defines where the source code is located.

- Git Repository
- Subversion
- Local Directory

2. STRATEGY

Defines how the build is executed.

- Docker
- Source-to-Image (S2I)
- Custom

3. TRIGGERS

Defines what starts a new build.

- Webhook
- Image Change
- Git Commit
- Manual

4. ENVIRONMENT

Defines build-time variables and environment settings.

- Environment Variables
- Build Arguments
- Secrets / Config Maps

5. OUTPUTS

Defines what is produced by the build.

- Docker Image
- Image Stream Tag
- Build Artifacts (Optional)

6. GENERAL

Additional build configuration settings.

- Name / Labels
- Service Account
- Timeouts
- Resource Limits

BUILD FLOW

1. **Trigger** — A trigger starts the build.
2. **Fetch Source** — Source code is retrieved.
3. **Build Execution** — The strategy runs the build.
4. **Produce Outputs** — Artifacts / images are created.
5. **Store Results** — Outputs are stored and ready to use.

COMMON TRIGGERS

- **Webhook** — Triggered by HTTP request.
- **Image Change** — Triggered when a base image changes.
- **Git Commit** — Triggered by changes in the source repo.
- **Scheduled** — Triggered on a schedule.
- **Manual** — Triggered manually by a user.

HIGH AVAILABILITY CONCEPTS

High Availability (HA) ensures applications and the platform remain available and resilient to failures, by eliminating single points of failure.

KEY BENEFITS

Minimize Downtime — keep applications and platform available even during failures.

Business Continuity — ensure critical workloads continue running without interruption.

Resilience — automatic recovery from failures and self-healing.

AVAILABILITY TARGETS

Level	Downtime/Yr	Availability
Standard	~24 hours	99.0%
High Availability	~4 hours	99.95%
Highly Available (HA)	~52 minutes	99.99%
Mission Critical	~5 minutes	99.999%

HA PILLARS

Redundancy **Distribution** **Failure Detection**

Automated Recovery

This is the direct enterprise reason behind Lab 42's own replicas: 2 requirement: a single Pod is a single point of failure -- two replicas, correctly labeled and probed, mean the Service keeps serving even if one Pod is lost.

FAILURE SCENARIOS & HOW HA HELPS

Worker Node Failure — Pods on the failed node are rescheduled onto healthy nodes.

Control Plane Node Failure — traffic routes to the remaining healthy control plane nodes.

etcd Node Failure — quorum is maintained by the remaining members -- no data loss.

KEY TAKEAWAY OpenShift and Kubernetes are built for High Availability across every layer -- redundancy, distribution, and automated recovery keep applications available through failures and maintenance.

HIGH AVAILABILITY CONCEPTS

High Availability (HA) is the ability of a system to remain operational and accessible for a desired level of performance during failures or maintenance.

KEY PRINCIPLES OF HIGH AVAILABILITY

NO SINGLE POINT OF FAILURE

Eliminate components whose failure can stop the system.

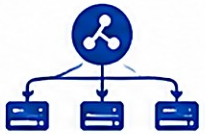
REDUNDANCY

Use duplicate components (resources, nodes, links) to take over in case of failures.

FAILOVER

Automatically switch to a healthy backup component when a failure occurs.

LOAD BALANCING

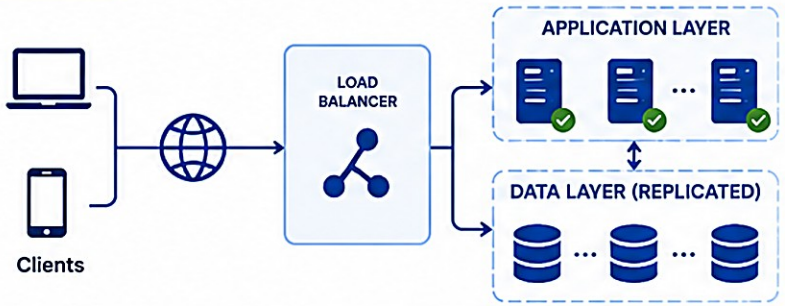


Distribute traffic across multiple instances to improve availability and performance.

MONITORING & ALERTING

Continuously monitor system health and alert on issues before impact.

HIGH AVAILABILITY ARCHITECTURE



Health Checks
Detect failures quickly

Automatic Recovery
Restore service without manual intervention

Data Replication
Keep data consistent and available

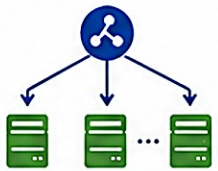
COMMON HA PATTERNS

ACTIVE–PASSIVE

Standby takes over when active fails.

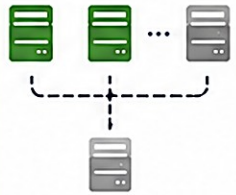
ACTIVE–ACTIVE

Multiple active nodes share the load.



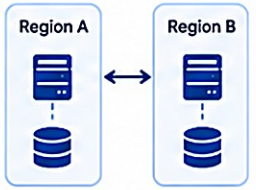
N+1 REDUNDANCY

Extra node(s) keep the system running if one fails.



GEOGRAPHIC REDUNDANCY

Services are replicated across regions.



BENEFITS OF HIGH AVAILABILITY

Maximized Uptime
Minimize downtime and outages

Improved Reliability
Consistent service for users

Better Performance
Handle more load and scale

Disaster Resilience
Recover from failures and disasters quickly

Business Continuity
Maintain operations and protect revenue

SELF-HEALING APPLICATIONS

The platform continuously monitors your applications and automatically takes corrective action to keep them running as desired -- no manual intervention.

HOW SELF-HEALING WORKS

COMMON MECHANISMS

Pod Failure Recovery — a crashed Pod is automatically restarted or recreated.

Node Failure Recovery — Pods on a failed node are rescheduled to healthy nodes.

Liveness & Readiness Probes — detect unhealthy Pods and prevent traffic to unready ones.

Rolling Updates & Rollbacks — automatic rollback if a new version causes failures.

crm-api probes (real Lab 42 pattern)

```
startupProbe:
  httpGet: { path: /actuator/health/liveness }
  failureThreshold: 30
readinessProbe:
  httpGet: { path: /actuator/health/readiness }
livenessProbe:
  httpGet: { path: /actuator/health/liveness }
```

Three distinct probes, three distinct jobs: startupProbe covers a slow PostgreSQL-dependent boot without triggering false restarts; readinessProbe gates traffic -- if it fails, the Pod is pulled from the Service's Endpoints but NOT restarted; livenessProbe is reserved for a truly wedged process and restarts the container. Pointing liveness at readiness (a real Lab 42 troubleshooting pitfall) causes restart storms during a normal DB blip instead of simply, safely shedding traffic.

KEY TAKEAWAY Self-healing means continuous monitoring plus automatic corrective action -- from a crashed Pod to a failed node -- keeping applications available, reliable, and resilient without manual intervention.

SELF-HEALING APPLICATIONS

Self-healing applications automatically detect, diagnose, and recover from failures without manual intervention, ensuring high availability and resilience.

KEY CAPABILITIES

AUTOMATIC DETECTION
Continuously monitor health and detect issues in real time.

DIAGNOSIS
Identify the root cause and determine the impact.

AUTOMATIC RECOVERY
Take corrective actions to restore the application automatically.

CONTINUOUS MONITORING
Verify the recovery and continue monitoring for new issues.

LEARN & IMPROVE
Learn from incidents and improve future healing actions.

SELF-HEALING WORKFLOW

1. MONITOR	2. DETECT	3. DIAGNOSE	4. HEAL	5. VERIFY
Monitor metrics, logs, traces, and events.	Detect anomaly or failure condition.	Analyze and identify root cause.	Execute automated recovery actions.	Validate recovery and restore state.
Input: Metrics, Logs, Events, Traces	Output: Alert Raised	Output: Root Cause Identified	Actions: Restart, Scale, Replace, Reroute, etc.	Output: Service Healthy

Continuous Feedback Loop

COMMON HEALING MECHANISMS

RESTART
Restart failed containers, pods, or services.

RESCALE
Scale up/down to meet demand or replace unhealthy instances.

REROUTE
Route traffic away from unhealthy instances or nodes.

REPLACE
Replace failed components with new healthy ones.

RECOVER DATA
Recover from data errors or loss using backups or replication.

BENEFITS

HIGH AVAILABILITY
Minimize downtime and service disruptions.

RESILIENCE
Maintain stability in dynamic environments.

OPERATIONAL EFFICIENCY
Reduce manual effort and speed up recovery.

COST OPTIMIZATION
Optimize resource usage and reduce incident cost.

BETTER USER EXPERIENCE
Ensure consistent performance and reliability.

BUSINESS CONTINUITY
Protect business processes and revenue.

TRY IT YOURSELF — EXERCISE 3: DESIGN THREE PROBES

Reinforces: the startup/readiness/liveness distinction from Self-Healing Applications, before Lab 42 configures them for real.

STEPS (notes/lab42-probe-design.md)

Step	What To Do
1 — Definitions	Write one sentence each: startup (slow boot), readiness (take traffic), liveness (restart if wedged).
2 — Check the Reference	Don't point all three at the same shallow endpoint -- readiness should reflect the real DB dependency.
3 — Paths	Propose Actuator paths/ports for each probe (placeholders are OK).
4 — Failure Story	Describe what agents/users see if readiness fails while liveness stays up.

Reference shape (from Self-Healing Applications)

```
startupProbe:   { path: /actuator/health/liveness,  failureThreshold: 30 }
readinessProbe: { path: /actuator/health/readiness }        livenessProbe:  { path: /actuator/health/liveness }
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-probe-design.md (workspace: examples/module-42-exercises).

AUTO SCALING FUNDAMENTALS

Automatic adjustment of application resources based on demand -- ensuring performance, availability, and cost efficiency without manual intervention.

KEY BENEFITS

High Availability — handle traffic spikes and maintain responsiveness.

Cost Efficiency — scale down during low demand to save resources.

Operational Simplicity — scaling decisions are automated based on real-time metrics.

HORIZONTAL VS VERTICAL

Autoscaler	What It Adjusts
Horizontal (HPA)	Number of Pod replicas -- best for stateless apps like crm-api.
Vertical (VPA)	CPU/Memory requests and limits of existing Pods -- no replica change.

HPA scaling on memory (a second metric)

```
spec:
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: memory
        target: { type: Utilization, averageUtilization: 80 }
```

USEFUL COMMANDS

Command	Does
kubectl get hpa	List all HPAs
kubectl describe hpa <name>	Detailed HPA status
kubectl top pods	CPU/Memory usage (needs metrics-server)

KEY TAKEAWAY Auto scaling ensures your applications automatically adapt to changing demand -- delivering performance, high availability, and cost efficiency with minimal operational effort.

AUTO SCALING FUNDAMENTALS

Auto Scaling automatically adjusts the number of application resources (e.g., server instances, pods) in response to changes in demand to maintain performance and optimize cost.

1. WHAT IS AUTO SCALING?

Automatically increases or decreases the number of running instances based on demand or defined policies.

2. HOW IT WORKS

Monitor
Collect metrics in real time

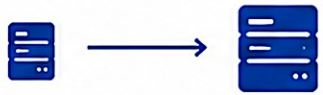
Evaluate
Compare metrics to scaling policies

Act
Add or remove instances as needed

Stabilize
Continue monitoring and adjust as needed

3. SCALING TYPES

Vertical Scaling (Scale Up/Down)



Increase or decrease the size (CPU, memory) of an instance.

Horizontal Scaling (Scale Out/In)

Increase or decrease the number of instances.

4. SCALING TRIGGERS

- CPU Utilization
- Memory Utilization
- Network In / Out
- Request Count / QPS
- Custom Metrics
- Scheduled Scaling

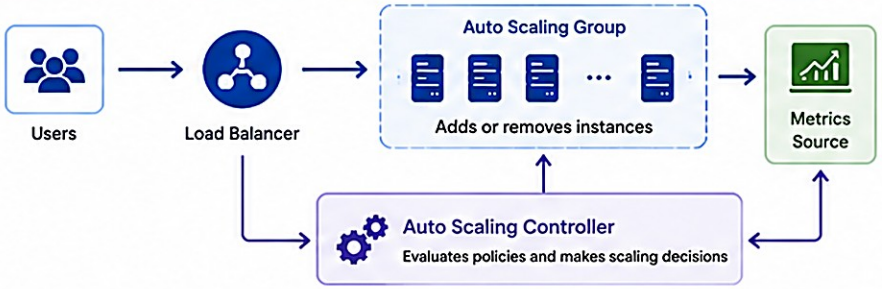
5. SCALING POLICIES

Target Tracking
Maintain a specific metric at a target level

Step Scaling
Scale by a fixed number based on metric ranges

Scheduled Scaling
Scale based on time or schedule

6. AUTO SCALING ARCHITECTURE



7. BENEFITS

High Availability
Handles traffic spikes and reduces downtime

Performance
Maintains optimal performance under varying loads

Cost Optimization
Scale down during low demand to save costs

Resilience
Automatically recovers from failures and changes

Flexibility
Adapts quickly to changing business needs

BEST PRACTICES

- Choose the right metrics that reflect user experience
- Set appropriate min, max, and desired capacity
- Avoid flapping with cooldown and stabilization settings
- Test scaling policies under realistic loads
- Monitor and refine policies continuously
- Use multi-metric and predictive scaling when possible

TRY IT YOURSELF — EXERCISE 6: RUNBOOK OUTLINE

The final pre-lab gate: outline docs/deployment-runbook.md so a peer could apply, verify, and roll back from it alone.

STEPS (notes/lab42-runbook-outline.md)

Step	What To Do
1 — Headings	Prereqs, apply order, verify probes, smoke CRM, rollback, contacts.
2 — Apply Order	Propose the real order: ConfigMap → Secret (out-of-band) → Deployment → Service → Ingress.
3 — Safety	Add a 'stop before destructive actions -- instructor approval' note.
4 — Scope	Mark this as an outline -- the full apply/smoke/rollback is real Lab 42 work.

Apply order to outline

```
1. ConfigMap  2. Secret (out-of-band)  3. Deployment  4. Service  5. Ingress / Route
Stop before any destructive action -- get instructor approval first.
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before continuing: exercises/exercise-06-runbook-outline.md. This is the final gate before Lab 42 begins.

LAB OVERVIEW -- KUBERNETES (K3S) DEPLOYMENT

Lab 42: deploy the CRM declaratively -- Deployment, Service, ConfigMap, Secret references, probes, Ingress, rollout, and a rehearsed rollback.

BUSINESS SCENARIO

- Leadership freezes: no shared-namespace deploy without probes, resource bounds, non-root policy, and a rehearsed rollback drill.
- You own that gate for the CRM API serving Amina (CUS-1001, ACTIVE) and Ravi (CUS-1002, PROSPECT) in the instructor project/namespace.
- Depends on Lab 41: you need a published or cluster-pullable crm-api image (version/digest) before starting.

LEARNING OBJECTIVES

- Author valid Deployment, Service, ConfigMap, and Ingress manifests.
- Separate non-secret config from Secret references.
- Set realistic CPU/memory requests and limits; implement three distinct probes.
- Verify rollout status, endpoints, events, and logs.
- Rehearse a bad revision and execute rollback to a known-good revision.
- Keep kubeconfig and Secret data out of source control; document steps in a runbook.

WHAT YOU BUILD

- lab42-crm/k8s/ manifests against the shared training k3s cluster: ConfigMap + out-of-band Secret reference, a non-root Deployment with resource limits and startup/readiness/liveness probes, a ClusterIP Service, and a Traefik Ingress with TLS redirect; deployed, smoke-tested with CUS-1001, deliberately broken and rolled back, and documented in docs/deployment-runbook.md.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation lab-request-001 traces every request; never paste kubeconfig or Secret data into Git or screenshots.

LAB TASKS AND DELIVERABLES

lab42-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Check cluster context, namespace, and image pull reachability before writing any manifest.
2	Create ConfigMap (crm-api-config) and a Secret reference created out-of-band (crm-api-secrets).
3	Define the Deployment: labels matching the selector, image, ports, envFrom ConfigMap + Secret.
4	Set resources (requests/limits) and a non-root securityContext.
5	Configure distinct startup, readiness, and liveness probes.
6	Create the ClusterIP Service and a Traefik Ingress (or Route) with TLS.
7	Apply, run rollout status, and diagnose from get pods/svc/endpoints, describe, logs, and events.
8	Smoke test: health readiness endpoint, then CUS-1001 via the Ingress with correlation lab-request-001.
9	Rehearse a bad revision in a sandbox, then rollout history + rollout undo; write the runbook.
10	Confirm 2-replica behavior: both Pods in Endpoints; delete one and observe minimal-downtime recovery.
11	Have a peer apply strictly from docs/deployment-runbook.md alone and reach a working rollback.

EXPECTED DELIVERABLES

- k8s/configmap.yaml, deployment.yaml, service.yaml, and route.yaml or ingress.yaml.
- secret.example.yaml with placeholders only -- no real values.
- Probe config with distinct startup/ready/live paths.
- Rollout success + rollback rehearsal evidence.
- CRM smoke evidence (CUS-1001, correlation).
- docs/deployment-runbook.md; no kubeconfig/Secret data in Git.

RUBRIC (100 MARKS)

- Environment/Structure 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

KEY TAKEAWAY Password in ConfigMap or Git is an honor violation. A single combined probe without rationale loses core marks. No rollback drill loses failure-handling marks.

MODULE 42 SUMMARY

In this module, you learned to deploy, expose, configure, scale, and safely roll back containerized applications with Kubernetes (k3s) -- and how OpenShift extends it.

KEY CONCEPTS COVERED

Kubernetes Fundamentals

Control plane, worker nodes, Pods, ReplicaSets, and Deployments.

Workloads & Networking

Deployments, Services, and the full deployment workflow.

Config & Secrets

ConfigMaps for non-secret data; Secrets for sensitive data.

Rollouts & Rollbacks

Rolling updates, safe scaling, and rehearsed rollback.

k3s & OpenShift

k3s fundamentals and OpenShift's Projects, Routes, Image Streams, Builds.

Hands-on Lab

Built lab42-crm: manifests, probes, Ingress, rollout, and rollback.

MODULE FLOW RECAP

1. **Manifest (YAML)**
2. **Deployment / RS**
3. **Pods**
4. **Service / Ingress**
5. **Lab: lab42-crm**

KEY TAKEAWAY Kubernetes automates deployment, scaling, and recovery. Proper labels, probes, resource limits, and a rehearsed rollback are what make a deployment production-ready.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Pods, ReplicaSets, Service types, and where sensitive data belongs.

1. What is the smallest deployable unit in Kubernetes?

A. Container
B. Pod
C. Node
D. Namespace

2. Which controller ensures a specified number of identical Pod replicas are always running?

A. Deployment
B. Service
C. ReplicaSet
D. ConfigMap

3. Which Service type is accessible only within the cluster by default?

A. NodePort
B. LoadBalancer
C. ExternalName
D. ClusterIP

4. Where should a database password be stored for a Pod to use?

A. ConfigMap
B. Secret
C. Directly in the Pod spec
D. Baked into the container image

KEY TAKEAWAY A Pod is the smallest deployable unit; a ReplicaSet maintains the desired replica count; ClusterIP is internal-only by default; passwords always belong in a Secret, never a ConfigMap.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on rolling update strategy, rollback commands, k3s internals, and probe design.

5. In a Deployment's RollingUpdate strategy, which field controls how many EXTRA Pods can be created above the desired count during an update?

A. maxUnavailable

B. maxSurge

C. replicas

D. minReadySeconds

6. Which command rolls a Deployment back to its previous revision?

A. kubectl rollout undo deployment/<name>

B. kubectl delete deployment/<name>

C. kubectl scale deployment/<name> --replicas=0

D. kubectl apply -f old.yaml --force

7. What datastore does k3s use by default for a single-node cluster?

A. etcd

B. SQLite

C. PostgreSQL

D. MySQL

8. Which probe should determine whether a Pod receives traffic, WITHOUT restarting it on failure?

A. Startup probe

B. Liveness probe

C. Readiness probe

D. Restart probe

KEY TAKEAWAY maxSurge caps extra Pods during a rollout; rollout undo reverts a Deployment; k3s defaults to SQLite; the readiness probe gates traffic without ever restarting the container.

"You declare the desired state; Kubernetes makes it happen."

MODULE 42 COMPLETE

Kubernetes (k3s) Deployment

You can now deploy, expose, configure, scale, and safely roll back containerized applications on Kubernetes (k3s) -- and compare architectures with OpenShift.